

Politecnico di Torino

Politecnico di Torino

Master's Degree in Computer Engineering

A.Y. 2025/2026

Graduation Session December 2026

GroundOne: A Specialist System One for Groundedness Verification in Local Retrieval-Augmented Generation

Fine-tuning compact language models as single-pass, calibrated verifiers on Apple Silicon

Supervisor: Prof. Paolo Garza

Candidate: Francesco Pio De Dominicis

Summary

Retrieval-augmented generation (RAG) is the standard way to give language models access to private documents, and running the whole pipeline on the user's own device is now practical on consumer hardware, which helps with privacy, offline use and cost. Retrieval does not, however, eliminate hallucination: generated answers still contain sentences that the retrieved context does not support or that contradict it. Detecting them is a prerequisite for trustworthy local assistants, and since the check must run on every answer, it has to fit on the same constrained machine as the generator.

Locality is the backbone of this work, and it has a precise technical consequence. On a single device, reading text is cheap and writing it is expensive: the model reads its input in one compute-bound pass, but generates its output one token at a time, each token limited by the memory bandwidth needed to read all the weights. On the laptop used here, writing a token costs fourteen to sixteen times more than reading one, and a review of current local hardware, from phones and laptops to dedicated machines such as Apple's Mac Studio and NVIDIA's DGX Spark, shows the same asymmetry everywhere, with new hardware accelerating reading much more than writing. Small open-weight models now trail the closed frontier by only months, so capable local pipelines are realistic; the question is how to verify them at a cost the device can bear.

Verification currently follows two paradigms. **Encoder-based checkers**, such as AlignScore, MiniCheck and HHEM, are small and fast, but return one fixed score per pair of document and sentence. **Generative judges**, by contrast, are flexible but write their verdicts token by token, so their cost grows with what they write; small judges often break the output format, and the confidence they express is rarely usable as a probability. A third family has recently emerged: **System One models**, such as the proprietary Jev and the open SemIf and Rizzo-Flow, answer closed questions in a single forward pass by reading the probability of each answer option from the logits. The evidence for them is mostly reported by their authors, and the open implementations are general-purpose. Whether a compact System One specialised in one task can match generative judges and dedicated encoders, where its advantage comes from, and whether its probabilities are usable,

all on consumer hardware, had not been studied in a controlled way.

This thesis designs, trains and evaluates **GroundOne**, a System One specialised in one decision: for each sentence of a RAG answer, whether the retrieved document supports it, contradicts it or does not mention it. A marker follows every sentence of the answer, and the verdicts for all K sentences are read from the logits of three answer letters in one forward pass, without generating any text. Qwen2.5 models with 0.5B and 3B parameters are fine-tuned with LoRA on the human-labelled answers of RAGTruth, with a class-weighted loss restricted to the three letters; a temperature and a bias fitted on development data calibrate the probabilities, and a threshold chosen on the same data turns them into consistent labels and strict or lenient flags. Everything, from building the dataset to training, evaluation and serving, runs on one Apple M4 laptop with 16 GB of memory, with MLX and no external service, and every baseline is run on the same machine. The evaluation uses three test sets built and frozen before training (the RAGTruth test split, LLM-AggreFact and VitaminC) and compares GroundOne with cues that never read the document, the untrained models, generative judges of the same size with and without probabilities, two general-purpose System One models and specialised encoders, reporting cluster-bootstrap intervals.

On the RAGTruth test set GroundOne reaches a three-way balanced accuracy of 0.769 with 0.5B parameters and 0.832 with 3B, against 0.37 and 0.44 for the untrained models; its AUROC (0.905 and 0.940) exceeds that of the encoders (about 0.82), and it clearly outperforms both general-purpose System One models: RizzoFlow, eight times larger than the 0.5B, and Liquid AI's d1-3B, of the same size as the 3B (0.657), which in turn is much stronger on the out-of-domain VitaminC test. On LLM-AggreFact the 3B is on par with MiniCheck-RoBERTa (74.8 against 74.0), above the published zero-shot 7B model and below the closed System One Jev as measured independently (78.6). When an answer contains fourteen or more sentences ($K \geq 14$), GroundOne is about two times faster than a terse judge and six times faster than a verbose judge of the same size and, unlike the judges, always produces a valid output. It also uses less energy (5.5 joules per answer with the 0.5B, 33 with the 3B), but the saving is smaller than the saving in time, because a long prefill draws more power than decoding. Combining vendor specifications with the measured length of the judges' outputs, a 3B judge must spend at least 1.8 seconds only on writing its verdicts for a long answer on the thesis laptop, and 0.12 seconds even on the fastest consumer graphics card: a cost that GroundOne does not pay on any hardware. Asked for the same output, generative judges provide probabilities that barely separate correct from incorrect decisions, while calibration brings GroundOne's expected calibration error on the test set to 0.019. How the decision is read also matters: with the same trained model, comparing only the three admissible letters is more accurate than letting the model generate its verdict freely, with a gain in balanced accuracy of 0.25 for the 0.5B and 0.07 for

the 3B. Some aspects remain to be improved, and they point to natural directions for future work: performance drops on domains far from the training data, such as the fact checking of VitaminC; the subtlest hallucinations, hidden in sentences that otherwise follow the source faithfully, are detected only in part; and training is not yet fully stable across random seeds, which simple changes to the optimiser could mitigate.

Acknowledgements

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Tristique senectus et netus et malesuada fames ac turpis. Pretium nibh ipsum consequat nisl vel pretium lectus quam. Urna molestie at elementum eu facilisis sed odio morbi quis. Sed egestas egestas fringilla phasellus faucibus scelerisque eleifend. At in tellus integer feugiat. Mauris rhoncus aenean vel elit scelerisque mauris pellentesque pulvinar. Commodo sed egestas egestas fringilla. Nunc lobortis mattis aliquam faucibus purus in massa. Facilisis magna etiam tempor orci eu lobortis elementum nibh. Elementum curabitur vitae nunc sed velit dignissim. Neque volutpat ac tincidunt vitae. Massa id neque aliquam vestibulum morbi blandit cursus risus at. Porta non pulvinar neque laoreet suspendisse interdum consectetur. Turpis in eu mi bibendum. Ut tristique et egestas quis ipsum suspendisse. Integer quis auctor elit sed vulputate mi sit amet. Viverra nam libero justo laoreet sit amet cursus.

Table of Contents

List of Figures x

1 Introduction 1
- 1.1 Motivation . . . 1
  - 1.1.1 Retrieval-augmented generation on the user's device . . . 1
  - 1.1.2 Hallucination and the need for verification . . . 2
  - 1.1.3 The cost of checking an answer on a laptop . . . 2
- 1.2 System One models . . . 3
- 1.3 Approach and research questions . . . 5
- 1.4 Scope . . . 6
- 1.5 Contributions . . . 6
- 1.6 Main results . . . 7
- 1.7 Thesis structure . . . 8

2 Local Inference of Language Models 9
- 2.1 From hosted to local models . . . 9
- 2.2 Open-weight and small language models . . . 12
  - 2.2.1 Open weights are not open source . . . 12
  - 2.2.2 The gap with closed models is closing . . . 13
  - 2.2.3 Small language models . . . 13
- 2.3 The cost of inference on one device . . . 15
  - 2.3.1 Prefill and decode . . . 15
  - 2.3.2 The roofline model . . . 16
  - 2.3.3 A lower bound on generation . . . 16
- 2.4 Hardware for local inference . . . 17
  - 2.4.1 Classes of hardware . . . 18
  - 2.4.2 The asymmetry holds on every class . . . 21
- 2.5 Runtimes and quantisation . . . 24
  - 2.5.1 Runtimes . . . 24
  - 2.5.2 Specialised inference engines . . . 24
  - 2.5.3 Quantisation . . . 26

2.5.4 Faster generation without fewer tokens . . . 27
2.6 Energy . . . 27
2.7 Consequences for verification on the device . . . 27

3 Background and Related Work 29
3.1 System One models . . . 29
3.2 Retrieval-augmented generation and hallucination . . . 31
3.3 Groundedness verification . . . 31
3.4 Calibration and selective prediction . . . 32
3.5 Parameter-efficient fine-tuning . . . 33
3.6 Research gap . . . 33

4 GroundOne: Method and Implementation 34
4.1 Task and decision . . . 34
4.2 Reading all verdicts in one forward pass . . . 35
4.3 Training objective . . . 36
4.4 Calibration . . . 37
4.5 Software . . . 38

5 Data 39
5.1 Task and labels . . . 39
5.2 Source datasets . . . 40
5.3 Construction . . . 41
5.4 The resulting dataset . . . 42
5.5 Reliability of the test sets . . . 43
5.6 Exploratory analysis . . . 45
5.7 Leakage control . . . 47

6 Experimental Setup 48
6.1 Models . . . 48
6.2 Baselines . . . 49
6.3 Metrics . . . 51
6.4 Statistical protocol . . . 51
6.5 Latency, energy and hardware . . . 52

7 Results 54
7.1 Training . . . 54
7.2 Variability across seeds . . . 54
7.3 Accuracy on the RAG test set (T1) . . . 56
7.4 Comparison with the baselines . . . 57
7.5 Transfer to a different domain (T3) . . . 59
7.6 Readout versus generation . . . 60

7.7 Calibration and selective prediction 60
7.8 Probabilities at equal output 62
7.9 Latency as a function of K 64
7.10 Energy per verification 65
7.11 Error analysis 66

8 Discussion, Limitations and Threats to Validity 67
8.1 When a specialist System One pays off 67
8.2 Limitations 69
8.3 Threats to validity 70

9 Conclusion and Future Work 72
9.1 Summary of findings 72
9.2 Future work 73

A Output of every system on one answer 74
A.1 Verdicts sentence by sentence 75
A.2 Raw outputs 76

Bibliography 78

List of Figures

1.1 Two ways of verifying the K sentences of an answer. Parallelograms: inputs and outputs; blue rectangles: components of the model; yellow rectangle: processing outside the model; diamonds: decisions. The System One reads the document and the answer once and, for every sentence, chooses among three admissible answers, also returning their probability. The generative LLM passes through the model again for every token it writes (red loop), and the verdicts must then be extracted from the text, which may not follow the required format. 4

2.1 Capability of the best closed and the best open-weight model over time on Epoch AI's aggregate capabilities index (dots: individual models; lines: best so far). Plotted from Epoch AI's published data table [34]. 14

2.2 Time per token when the model reads its input (prefill) and when it writes its output (decode), measured on the thesis machine (Apple M4, 16 GB, MLX 8-bit, batch 1). Prefill: median over contexts of 500, 1,000 and 2,000 tokens. 15

2.3 Roofline model of the thesis machine [41]. Sloped roof: Apple's memory bandwidth (120 GB/s). Flat roof: throughput reached in prefill, a lower bound of the unpublished peak. Intensities computed for one feed-forward layer of Qwen2.5 with 8-bit weights and 16-bit activations; attention is ignored. 17

2.4 Memory needed for the weights of a model as a function of its size and precision (parameters times bits per weight, including half a bit of per-group scales for the quantised formats). Dashed lines: common memory sizes of local hardware. The key-value cache, activations and other running models come on top. 18

2.5 The two architectures of local inference hardware. Blue rectangles: compute units; green rectangles: memory; orange rectangle: the model weights; line width: indicative bandwidth. Capacity and bandwidth values from Table 2.2. In a discrete system a model that does not fit in VRAM must go through the much slower PCIe link; in a unified-memory system the model can occupy almost all of the machine's memory, but every generated token reads it at a lower speed than a high-end graphics card. 19
2.6 Some of the local inference systems compared in this section (images not to scale). Sources: (a), (c) Apple; (b) LoMit, Wikimedia Commons, CC0; (d), (e) NVIDIA. The vendors' images are promotional material, reproduced for the purpose of scientific documentation. . . 20
2.7 Local inference hardware on the two axes that decide what can run: memory capacity (which models fit) and memory bandwidth (how fast tokens are generated). Vendors' specifications: markers at the largest memory configuration, shaded bars over the range of memory sizes offered; the thesis machine is shown at its actual 16 GB. 22
2.8 Lower bounds on the cost of generating text on each class of local hardware, computed as weight bytes divided by the vendor's bandwidth (one request at a time; key-value cache ignored). (a) Time per generated token with the two GroundOne base models in 8 bits; diamonds: decode time measured on the thesis machine. (b) Minimum time that a 3B generative judge spends only on writing the verdicts for an answer of fourteen or more sentences (median generated tokens of the latency protocol). A System One generates no token, so both costs are zero for it on every machine. 23
2.9 Streaming experts from the SSD in an engine such as Colibri. Memory holds the part of the model used by every token (attention, router, shared experts) and a cache of recently used experts; every expert is stored on the SSD. For each token the router chooses a few experts: those already cached are used at once, the missing ones are read from disk, at a bandwidth of a few gigabytes per second. For example, GLM-5.2 keeps about 10 GB resident and leaves about 429 GB of weights on disk [60]. The diagram is not to scale. 25

5.1 Class balance of the labelled sentences in each split. RAGTruth (train, development and T1) is about 88–91 % supported; T3 is sampled to 1,000 pairs per class. 43
5.2 Distribution of K, the number of scored sentences per answer, in training and in T1. Dashed lines mark the bands used to report results as a function of K. 44

5.3 Share of contradicted or not-mentioned sentences by the model that wrote the answer, in training and in T1. 45
5.4 Word overlap between a sentence and its document, by class, in training, T1 and T3 (mean in brackets). 46
5.5 Share of contradicted or not-mentioned sentences by position in the answer. 47

7.1 Three-way balanced accuracy on the development set during training of the 0.5B model, overall (left) and per task (right). The dotted line marks chance. The 3B shows the same shape on its 200-answer development subset. 55
7.2 Development three-way balanced accuracy of the 0.5B during training with the same recipe and three seeds. Two runs converge to the same level; the third collapsed to a constant predictor and was stopped. . 55
7.3 T1: AUROC of P(supported) for every system (left), and three-way balanced accuracy for the systems with a three-way output (right). Rizzo-Flow is evaluated on 300 answers and the untrained 3B on 600; all other systems, including d1-3B, on the full test set. 58
7.4 Control B2 on the full T1: with the same weights and the same forward pass, the verdict is read from the three answer letters (GroundOne) or taken as the token that greedy generation would emit (format failures counted as errors). 61
7.5 Reliability of P(supported) of the 0.5B before (raw) and after calibration on the development set, for the development set, T1 and T3. Points are bins with at least 20 sentences. 61
7.6 Risk–coverage curves of the binary decision of the 0.5B with calibrated probabilities: error rate as the least confident sentences are deferred. 62
7.7 Equal-output comparison on the 200 latency answers: generative judges of the same size asked for GroundOne's output, with probabilities read from the logits of the generated letter or written in JSON; all systems calibrated on development data with the same map. Sentences with a valid verdict only. 63
7.8 Median time per answer as a function of K on the same 200 T1 answers (in-process MLX, one answer at a time, M4 with 16 GB), for GroundOne, the System One with one question per sentence, and the terse and verbose generative judges. Logarithmic scale. . . 64
7.9 On the latency subset: three-way balanced accuracy (format failures counted as errors) and share of sentences whose verdict could not be parsed. 65

7.10 Energy per verified answer on 100 latency answers (powermetrics, CPU, GPU and Neural Engine, net of idle power), with the mean net power during the run. 65

Chapter 1

Introduction

1.1 Motivation

1.1.1 Retrieval-augmented generation on the user's device

Retrieval-augmented generation has become the standard way to let a language model answer questions about documents it was never trained on. A retriever selects the passages that are relevant to the question, and the model writes its answer with those passages in its context [1]. The same recipe serves a company searching its internal reports, a lawyer reading a contract and a student questioning their own notes, and in each case the value of the answer depends on its staying faithful to the documents.

Until recently such a pipeline required a hosted model, reached over the network. This is no longer the case. Compact open-weight models and efficient runtimes now run on consumer hardware [2, 3], and platform vendors ship models of a few billion parameters directly on their devices [4]. The gap with the best closed models is small and shrinking: models that fit on a single consumer graphics card reach the performance of the frontier of the time within six to twelve months [5], and the lead of the best closed model over the best open one on a popular leaderboard fell from 8 % to under 2 % in a single year [6]. The whole pipeline, from retrieval to answer, can therefore stay on the user's own machine. This matters for four reasons. When the documents are private or regulated, they never leave the device; privacy becomes a technical property of the system rather than a promise of a provider, which the user cannot verify. A recent dispute over credit between OpenAI and two mathematicians, who had written unpublished drafts inside one of the company's products, showed how real the question is: the provider itself stated that it could not rule out that data derived from that usage had helped improve its models [7]. When there is no network, the assistant keeps working. When the behaviour of the system must stay stable, the weights change only if the user changes them,

instead of following a provider's updates. Finally, the cost becomes that of the hardware and its energy, instead of a price per token. Local models also expose their logits, which hosted interfaces often hide or limit, and this property turns out to be essential for the approach studied here.

Locality is a constraint as well as an advantage. A laptop has limited memory, limited bandwidth and a battery, and every component of the assistant must fit within them at the same time. This thesis treats locality as a requirement rather than as an afterthought: every step of the work, from building the dataset to training, evaluation and serving, runs on one consumer laptop with 16 GB of memory, and no step depends on an external service. Because locality is the backbone of the work, Chapter 2 is devoted entirely to it: why it matters, which models and hardware make it possible, and what it costs.

1.1.2 Hallucination and the need for verification

Retrieval does not remove hallucination. Even with the right passages in context, generated answers still contain sentences that the passages do not support, or that contradict them [8, 9]. An example from the data used in this thesis shows how subtle the problem can be. Asked to describe a restaurant from a structured record, a model writes six fluent sentences. Four of them are correct. One states that the restaurant is open seven days a week, whereas the record shows that it is closed on Mondays: the sentence contradicts the source. Another states that the restaurant has a rating of 4.5 stars on Google, whereas the record gives the rating without naming any source: the sentence adds a detail that the source does not mention (Appendix A).

Two observations follow from this example. First, the error sits in a single sentence of an otherwise correct answer, so a verdict on the whole answer is of little help: the user needs to know which sentence to distrust. Second, the two erroneous sentences are wrong in different ways. A contradiction is a factual error with respect to the source, whereas an unsupported addition may be harmless background or an invention, and an application may want to treat the two differently. A useful verifier therefore decides, for every sentence, whether the document supports it, contradicts it or does not mention it. Since no user can be expected to check every sentence against its source, the verifier must run on every answer, automatically, and on the same machine that generated it.

1.1.3 The cost of checking an answer on a laptop

A verifier that runs next to the generator competes with it for the same memory, time and battery. The most common solution, asking a language model to judge the answer, is expensive precisely where a laptop is weakest. A judge writes its

verdicts token by token, and on local hardware each generated token costs far more than each token it reads. Reading the prompt is limited by computation, which a laptop GPU handles well, whereas writing the output is limited by memory bandwidth, because every new token must read all the weights [10]. On the laptop used in this thesis, reading one token takes about 0.37 milliseconds for a model of 0.5B parameters, while writing one takes about 5.9, sixteen times more, and the asymmetry holds on every class of local hardware, from phones to dedicated AI machines (Chapter 2). A judge that explains its reasoning before each verdict multiplies this cost by the length of its explanation, and the cost grows again with the number of sentences in the answer. Small judges also break the required output format, and the confidence they express is rarely usable as a probability.

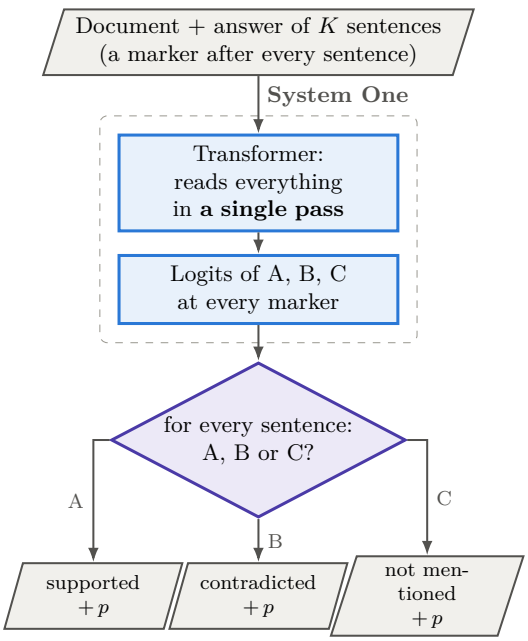
Encoder-based verifiers avoid generation altogether. Models such as AlignScore, MiniCheck and HHEM read a pair of document and sentence and return a score of support [11, 12, 13]. They are small and fast, but they give one fixed binary score per pair: they do not distinguish a contradiction from an omission, they read each sentence without the rest of the answer, and adapting them to a new decision requires a new model.

1.2 System One models

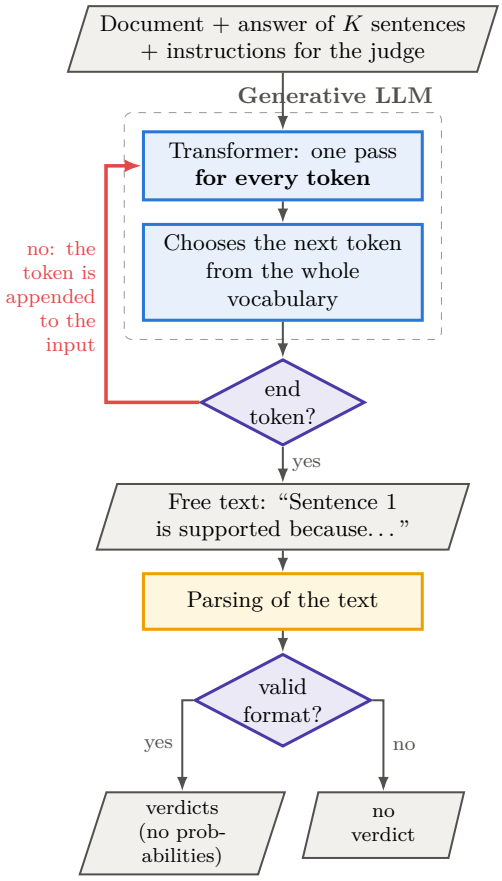
A different way to use a language model for decisions has recently attracted attention under the name of *System One models* [14]. The name evokes the fast, intuitive mode of thinking, as opposed to deliberate reasoning. Instead of generating an answer, such a model reads the question and the context once and returns a probability for each admissible answer, for example by reading the logits of the option letters at the end of the prompt (e.g., A: 90 %, B: 5 %, C: 5 %) [15, 16]. Three properties follow directly. No text is generated, so the decision costs only the reading of the input. The output is always one of the admissible answers, so it is always valid. Many questions about the same context can share a single pass over it, so each additional question costs little.

The idea fits verification well. An answer of K sentences poses K closed questions about the same document, and each of them has three admissible answers. A System One can answer all of them while reading the document only once, and the probability it returns for each sentence can be calibrated and compared with a threshold chosen by the application. A traditional LLM, instead, produces its answer autoregressively: after reading the document it must generate the verdicts one token at a time, with a new pass through the model for every token written, and only at the end can the text it produced be interpreted as a decision (Figure 1.1).

The field is young and moves quickly. After the first hosted model, open implementations followed [15, 16], and on 9 October 2026 the Hugging Face hub



(a) System One: one reading and, for every sentence, a choice among three admissible answers.



(b) Traditional LLM: one pass through the model for every token, until the text is complete.

Figure 1.1: Two ways of verifying the K sentences of an answer. Parallelograms: inputs and outputs; blue rectangles: components of the model; yellow rectangle: processing outside the model; diamonds: decisions. The System One reads the document and the answer once and, for every sentence, chooses among three admissible answers, also returning their probability. The generative LLM passes through the model again for every token it writes (red loop), and the verdicts must then be extracted from the text, which may not follow the required format.

officially introduced a tag for these *decision models*, which the following day already grouped about 1,500 models, more than half of them published in the previous weeks [17]. Established organisations released their own, such as Liquid AI's d1-3B and Cloudflare's Clef [18, 19]. The evidence, however, lags behind the releases. Most of it comes from the authors of each system, independent evaluations are recent and

often less favourable [20, 21], and a common leaderboard finds that general-purpose decision models with fewer than one billion parameters barely exceed the trivial baseline on hallucination detection in retrieval-augmented answers [22]. All the open implementations, moreover, are general-purpose systems that answer any typed question.

Three problems therefore remain open. The first is whether, for a single task, a small specialised System One is worth using. The second is how such a model compares with the alternatives: a general-purpose System One, a generative judge and a dedicated verifier. The third is where any advantage comes from: from the specialisation, from the way the decision is read or, more simply, from the size of the model.

1.3 Approach and research questions

This thesis designs, trains and evaluates GroundOne, a compact System One specialised in one decision: for every sentence of a RAG answer, whether the retrieved document supports it, contradicts it or does not mention it. The answer is written with a short marker after each sentence, and the verdicts of all sentences are read in one forward pass, from the logits of three answer letters at the markers, without generating any text. Qwen2.5 models with 0.5B and 3B parameters are fine-tuned with LoRA on human-labelled answers from RAGTruth [23, 24, 8], and their probabilities are calibrated on development data. Three test sets were built and frozen before any model was trained, so that no design choice could be tuned on them. The three open problems become five research questions: the first two restate them in a testable form, and the other three add what matters for real use on the device, namely the reliability of the probabilities, the cost and the value of a three-way decision.

RQ1, specialisation. Can a System One of 0.5B or 3B parameters, fine-tuned on the task, decide whether each sentence is faithful to the document? And does it do so as well as larger models used zero-shot[1] and as the encoders specialised in this kind of verification?

RQ2, readout versus generation. How much does the System One mechanism, that is, the readout[2] of the decision from the logits, contribute at the same

[1] A model is used *zero-shot* when it tackles a task without any task-specific training and without examples in the prompt: it receives only an instruction that describes the task.

[2] The *readout* is the reading of the decision directly from the logits of the model: instead of letting it write an answer, the scores it assigns to the admissible answers alone (here the letters A, B and C) are compared and the most probable one is chosen.

model size? The comparison is twofold: against generative judges that write their verdicts and against the same model left free to generate its answer. And how does the difference change as the number of sentences to verify grows?

RQ3, calibration. Are the probabilities returned by the model reliable as a measure of confidence? In particular, can they be used to set a decision threshold and to defer uncertain cases to the user? And are they better than those that can be obtained from a generative judge?

RQ4, cost on the device. Does the verifier fit within the memory, time and energy limits of a 16 GB laptop? And how much does it cost compared with a generative judge running on the same machine?

RQ5, beyond a binary score. What does a user of the verifier gain from a three-way decision for every sentence, compared with the plain score of a binary verifier?

1.4 Scope

The thesis studies the verification step in isolation. The retrieved document is given, so the quality of retrieval is not evaluated, and an answer is judged only against the document it was generated from, not against world knowledge. GroundOne flags sentences but does not correct them, and it returns verdicts and probabilities rather than written explanations. The data are in English. Documents longer than the context used in training are split into parts, as described in Chapter 6. Finally, the experiments target one class of hardware, a consumer laptop with unified memory, and every system, including every baseline, runs on the same machine.

1.5 Contributions

The thesis makes four contributions.

1. **GroundOne**, an open specialist System One for sentence-level groundedness verification that runs entirely on a laptop. Thanks to a marker placed after every sentence, it reads all the verdicts of an answer in one forward pass. It is released as an installable package with a command-line interface and a local HTTP API, which also includes an interface compatible with existing System One clients.

2. **A sentence-level dataset derived from public annotations.** No label was annotated anew: the human annotations of RAGTruth, which mark spans of text, were converted into a three-way label for every sentence, with

rules fixed in advance. The three test sets come from public benchmarks, were checked automatically, for instance on sentence segmentation, and were frozen before any model was trained, with leakage controlled at the level of documents.

3. **A controlled decomposition of the gains.** The comparison isolates the contribution of specialisation, of the restricted readout and of model size. GroundOne is compared with cues that never verify a sentence, untrained models, generative judges with and without probabilities, two general-purpose System One models (Rizzo-Flow, with 4B parameters, and Liquid AI's d1-3B, of the same size as the 3B) and specialised encoders, all on the same hardware and with intervals obtained by cluster bootstrap.

4. **A study of calibration, time and energy**: how the probabilities can be calibrated and turned into consistent labels, how much they help in deferring uncertain decisions, and how the time and energy of a verification grow with the length of the answer. The study is set in an analysis of local inference that combines vendor specifications with own measurements to bound, for each class of consumer hardware, what a verifier that generates text must pay and a System One does not.

1.6 Main results

The results show that specialisation works in the domain it was aimed at. On the RAGTruth test set GroundOne reaches a three-way balanced accuracy of 0.769 with 0.5B parameters and 0.832 with 3B, against 0.37 and 0.44 for the same models before training. It ranks supported sentences better than the specialised encoders, and it clearly outperforms both general-purpose System One models: Rizzo-Flow, eight times larger than the 0.5B, and d1-3B, of the same size as the 3B. On LLM-AggreFact, a benchmark drawn from different sources, the 3B is on par with the best small encoder.

The System One mechanism pays off in speed and in accuracy. When an answer contains fourteen or more sentences, GroundOne verifies it about two times faster than a terse judge and six times faster than a verbose judge of the same size, with an output that is always valid and with less energy. How the decision is read also matters: with the same trained model, choosing the most probable of the three admissible letters is more accurate than letting the model generate its verdict freely, with a gain in balanced accuracy of 0.25 for the 0.5B and 0.07 for the 3B. After calibration, the probabilities of GroundOne are close to the observed frequencies, whereas the probabilities of generative judges asked for the same output barely separate correct from incorrect decisions.

Some aspects remain to be improved, and they point to natural directions for future work. Performance drops on domains far from the training data, where the general-purpose System One d1-3B is stronger; the subtlest hallucinations, hidden in sentences that otherwise follow the source faithfully, are detected only in part; and training is not yet fully stable across random seeds[3].

1.7 Thesis structure

Chapter 2 is devoted to local inference: the reasons for keeping models on the device, open-weight and small models, the cost model of inference on one device, the hardware landscape, runtimes and quantisation, energy, and the consequence for verification. Chapter 3 reviews System One models and the decision-model ecosystem, hallucination in retrieval-augmented generation, verification methods and calibration. Chapter 4 describes GroundOne: the decision, the single-pass readout, the training objective, the calibration and the served output. Chapter 5 defines the labels and describes how the training and test sets were built, checked and frozen. Chapter 6 presents the trained models, the baselines, the metrics and the measurement protocol. Chapter 7 reports the results, Chapter 8 discusses what they mean and where they stop, and Chapter 9 concludes. Appendix A shows the output of every system on the example answer described in Section 1.1.2.

[3]The *seed* is the number that initialises the random number generator used during training, for instance to initialise the added weights and to shuffle the order of the data. Repeating the training with different seeds produces slightly different models: if the results change a lot from one seed to another, the procedure is not robust.

Chapter 2

Local Inference of Language Models

This thesis builds a verifier that runs entirely on the user's own machine, and this choice shapes every other one: the size of the models, the way decisions are read from them, the measurements and the comparisons. This chapter explains why local inference matters, what makes it possible today and what it costs. It starts from the reasons for keeping a model on the device and from the open-weight models that make this possible. It then derives a simple cost model of inference on one device, which shows why reading a token costs far less than writing one, and surveys the hardware on which local models run, from the laptop used in this work to dedicated desktop machines. It continues with the software and the quantisation that bring models within reach of that hardware, and with energy. It closes by drawing the consequence for verification, which motivates the design of GroundOne in the following chapters.

2.1 From hosted to local models

Most people use language models through hosted services. The user's input travels over the network to a provider, a model in a datacenter processes it, and the answer travels back. The model, the hardware and the policies that govern the data all belong to the provider. Running the model on the user's own device changes all of this at once. Five properties follow, and each of them matters for a retrieval-augmented assistant that works on private documents.

Privacy as a technical guarantee. With a hosted model, every document and every question is sent to a third party, and what happens to them afterwards, how long they are kept, who can access them and whether they are used to improve

future models, is governed by contract and policy. The user can read the policy but cannot verify that it is respected. With a local model the documents never leave the machine, so the question does not arise: privacy becomes a property of the system rather than a promise. Surveys of edge intelligence state the point directly: on-device inference eliminates privacy leakage and the need for an Internet connection [25].

European law makes the difference concrete. Under the General Data Protection Regulation, the processing of personal data explicitly includes its "disclosure by transmission" [26, Art. 4(2)]. Sending a document that contains personal data to a hosted model therefore makes the provider a processor, bound by a contract with the specific guarantees of Article 28, and when the provider operates outside the European Union the transfer must rest on an adequacy decision, on appropriate safeguards or on one of the derogations of Articles 44–49 [26]. A verifier that runs on the user's device involves no processor and no transfer. It does not remove every obligation, since the organisation that uses it must still respect the principles of Article 5 and secure the device, but it removes the obligations that depend on trusting someone else. Many confidential documents, such as unpublished research, contracts or source code, contain no personal data at all; for them the regulation does not apply, and the argument becomes one of confidentiality.

A recent episode illustrates what is at stake. In September 2026 OpenAI announced that an internal multi-agent system had produced a proof of blow-up for the forced Navier–Stokes equations, one of the Millennium Prize problems, formalised in the Lean proof assistant [7]. Two mathematicians, Tristan Buckmaster and Levent Alpöge, had released related results for the Euler equations at about the same time, and disputed the credit. Buckmaster had written his drafts in OpenAI's own coding assistant, and asked the company "whether the model had been trained on, or had access to, our sessions in Codex, into which we had been putting all our drafts" [27]. OpenAI denied that its researchers or agents had seen the work before its release, but added: "While unlikely, we cannot rule out that de-identified data derived from their usage of our products helped improve our models" [7, 28]. Nothing in the public record shows that the drafts were misused, and the dispute is mainly about credit. What the episode does show is that, once unpublished work is inside a hosted model, neither the user nor, by its own statement, the provider can exclude that it contributed to the provider's models. With a local model this question cannot be asked.

Providers are aware of the problem. Apple describes on-device processing as the cornerstone of its approach to privacy, and for requests that exceed the device it built Private Cloud Compute, a dedicated cloud with custom servers, no privileged access for staff, no persistent storage of requests and a public log of the software images that run on it [29]. The design is a serious attempt to make hosted inference verifiable, and its complexity is itself an argument: privacy in the cloud requires

custom silicon, a hardened operating system and cryptographic attestation, and it still rests on trust in the hardware, in the supply chain and in the attestation, which only security researchers, not users, can inspect [29].

Availability. A local model works without a network and without depending on the availability of a provider. For an assistant used on a train, in a hospital ward with restricted connectivity or in an air-gapped environment, this is a requirement rather than a convenience.

Stability and reproducibility. The weights of a local model change only when the user changes them. A hosted model can be updated, re-tuned or withdrawn at any time, often silently, so that the same request produces a different answer from one month to the next. For a verifier whose thresholds are calibrated on development data (Chapter 4), a silent change of the underlying model would invalidate the calibration without warning.

Cost. A hosted model is paid per token, and a verifier that checks every answer multiplies the number of tokens processed. A local model is paid once, with the hardware, and then costs only its energy.

Access to the model's internals. A local model exposes its logits, the scores from which every probability is computed. Hosted interfaces often hide them or return them only for a few tokens. The approach studied in this thesis reads the decision directly from the logits of a few answer letters and calibrates them, so it requires this access; it is, in this sense, possible only with a local model.

The price of locality. These advantages have a cost. A laptop holds only small models, and it must hold them next to the operating system, the user's applications and every other component of the assistant. It has less memory bandwidth and less computation than a datacenter accelerator, and it runs on a battery. Part of the literature concludes that pure on-device deployment is too limited and proposes to split the work with an edge server or with the cloud [25, 30]: a small local model handles easy requests and forwards hard ones, or drafts an answer that a larger remote model checks. Apple's combination of an on-device model with Private Cloud Compute is the most visible example [4, 29]. These hybrid designs reduce the cost of locality but give back part of its benefits, since whatever is forwarded leaves the device. This thesis takes the strict position: the whole verification stays on the device, and the rest of the chapter examines what this requires.

Table 2.1 summarises the comparison.

Table 2.1: Hosted and local inference compared on the properties that matter for a verifier of private documents.

Property	Hosted model	Local model
Privacy	Contractual: documents are sent to a processor; the user cannot verify how they are used.	Technical: documents never leave the device.
Regulation (GDPR)	Processor contract (Art. 28); third-country transfers (Arts. 44–49).	No processor and no transfer; Art. 5 principles and device security still apply.
Availability	Requires network and provider uptime.	Works offline.
Stability	The provider may update or withdraw the model.	Weights change only when the user changes them.
Cost	Price per token.	Hardware once, then energy.
Logits	Often hidden or limited.	Fully accessible: readout and calibration possible.
Capability	Frontier models.	Small models, limited by memory, bandwidth and battery.

2.2 Open-weight and small language models

Local inference requires a model whose weights can be downloaded. This section describes what such models are, how far they trail the best closed models, and which of them are small enough for a personal device.

2.2.1 Open weights are not open source

A model is called *open-weight* when its trained parameters are published and can be run and modified by anyone. This is not the same as open source. The Open Source Initiative defines an open-source AI system through the four freedoms of the free-software tradition, to use, study, modify and share the system, and requires that the form preferred for modifying it be available: the parameters, the complete code used to train and run it, and enough information about the training data for a skilled person to build a substantially equivalent system [31]. The data themselves need not be released, but weights alone do not qualify. Most models that are commonly called open publish their weights and an inference recipe, but neither the training code nor a detailed description of the data. Openness is therefore a scale rather than a binary property. An index that scores models on the availability

of their weights, the disclosure of their pre-training and post-training data, their methodology and their licence finds that no model reaches the maximum, that fully documented research models such as OLMo score high, and that some popular open-weight models publish little beyond the weights [32].

Licences add a further distinction. The two models fine-tuned in this thesis come from the same family but carry different licences: Qwen2.5-0.5B is released under the permissive Apache 2.0 licence, whereas Qwen2.5-3B is released under a research licence that restricts commercial use [23]. In the terminology above both are open-weight models, and the thesis uses this term for them. The distinction also matters for regulation: the European AI Act relaxes some transparency obligations for general-purpose models released under free and open licences, but not for models that pose systemic risk, which it presumes above 10^{25} floating-point operations of training compute [33].

2.2.2 The gap with closed models is closing

Local inference would be of limited interest if open models were far behind closed ones. They are not. On the Chatbot Arena leaderboard, where users compare the answers of two anonymous models, the lead of the best closed model over the best open-weight model fell from 8.04 % in January 2024 to 1.70 % in February 2025 [6]. On Epoch AI's capabilities index, which aggregates many benchmarks, the best open-weight models trailed the closed frontier by about four months on average between January and May 2026 (Figure 2.1) [34].

What matters for a personal device is not the best open model, which may have hundreds of billions of parameters, but the best model that fits on consumer hardware. Epoch AI tracks this quantity directly. Considering models small enough to run on a single consumer graphics card, at most about 28 billion parameters for a card with 24 GB and about 40 billion for one with 32 GB, it finds that across four benchmarks they reach the performance of the frontier model of the time after six to twelve months [5]. Three forces drive the catch-up: open developers scale their training compute about as fast as closed ones, distillation transfers much of the capability of a large model into a small one, and each new generation of graphics cards raises the size of the models that fit [35]. The same trend appears from the other side: the smallest model scoring above 60 % on the MMLU benchmark had 540 billion parameters in 2022 and 3.8 billion in 2024, a reduction of 142 times [6].

2.2.3 Small language models

At the low end of the scale are *small language models*. Definitions vary; a survey that measures several dozen of them on devices places the range at 100 million to 5 billion parameters, decoder-only [36]. Small models are trained on far more data

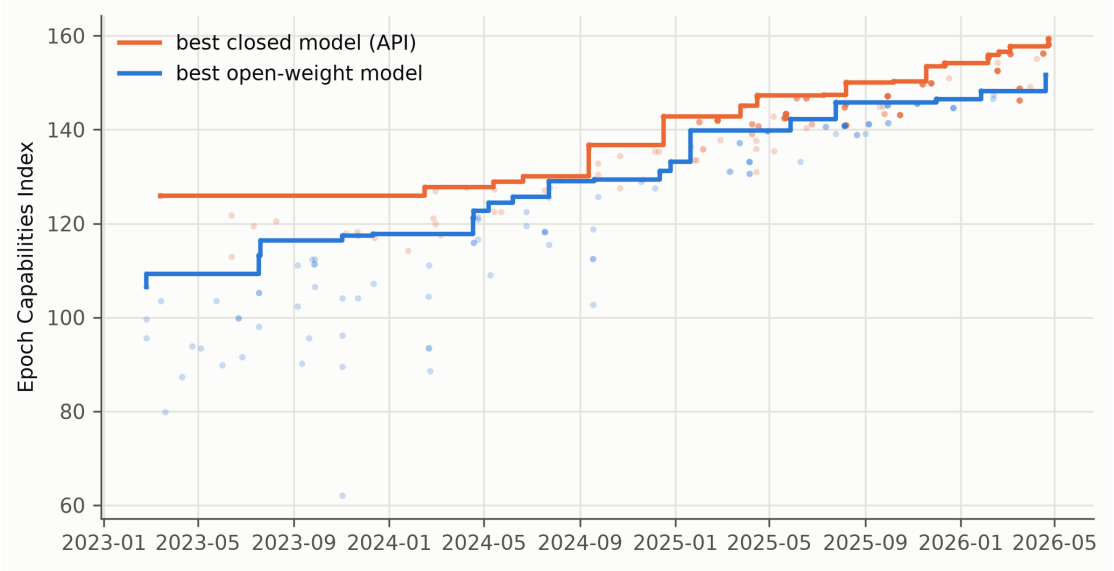


Figure 2.1: Capability of the best closed and the best open-weight model over time on Epoch AI's aggregate capabilities index (dots: individual models; lines: best so far). Plotted from Epoch AI's published data table [34].

than the compute-optimal recipe would suggest for their size, because the cost that matters for them is not the training but the inference that follows: a model of half a billion parameters may see trillions of tokens [36]. Measured on edge devices with 4-bit weights, models between 0.1 and 3 billion parameters need between 0.3 and 2.5 GB of memory, and at long context the key-value cache and the computation buffers take most of it [36]. Research on models below one billion parameters argues that, on phones, an application should not use more than about a tenth of the device's memory, and shows that a carefully designed 350-million-parameter model matches a general 7-billion-parameter model on a narrow task such as calling an application interface [37].

Platform vendors have adopted the same idea. Apple's foundation model for its devices has about three billion parameters, is quantised to under four bits per weight and is specialised for each feature with small low-rank adapters [4]; Google's Gemma 3n family is designed to run on phones and laptops with a reduced memory footprint, and is marketed explicitly for privacy and offline use [38]. GroundOne follows the same pattern: a small open-weight model, quantised, specialised by an adapter for one task.

2.3 The cost of inference on one device

The previous sections explain why a model should run locally and which models can. This section explains how much it costs to run one, and why the cost depends so strongly on what the model is asked to do.

2.3.1 Prefill and decode

A decoder-only language model processes a request in two phases. In the *prefill* phase it reads the whole prompt at once: every layer computes, for every input token, the keys and values that later tokens will attend to, and stores them in a key-value cache. In the *decode* phase it produces the output one token at a time, each new token attending to the cache and extending it by one position [10]. Prefill processes many tokens in parallel; decoding processes one. Serving systems exploit the difference in several ways, for instance by managing the cache in pages to fit more concurrent requests [39] or by running the two phases on different machines suited to each [40].

The consequence is visible on the laptop used in this thesis (Figure 2.2). With the 0.5B model, reading one token of the prompt takes less than 0.4 milliseconds, while writing one token of output takes 5.9 milliseconds, sixteen times more. With the 3B model the figures are 2.2 and 31.8 milliseconds, fourteen times more. A model that writes a short paragraph thus spends more time on it than on reading a document of several thousand tokens.

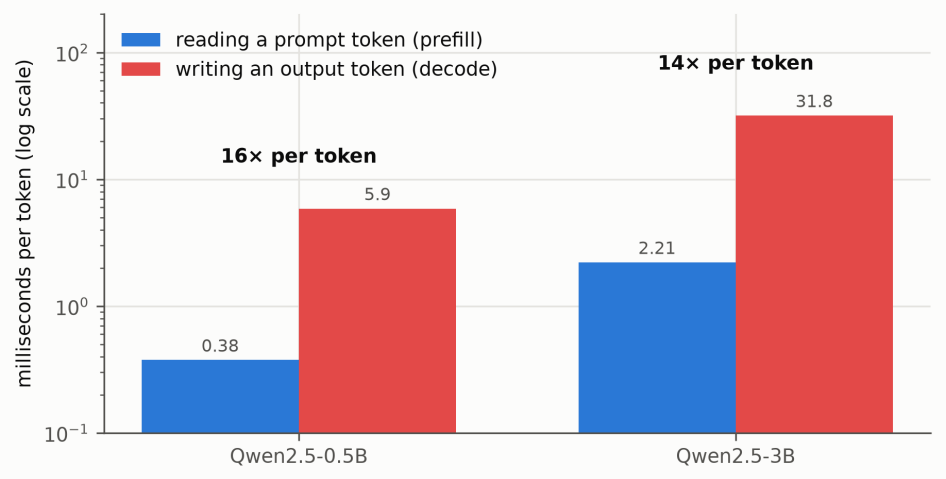


Figure 2.2: Time per token when the model reads its input (prefill) and when it writes its output (decode), measured on the thesis machine (Apple M4, 16 GB, MLX 8-bit, batch 1). Prefill: median over contexts of 500, 1,000 and 2,000 tokens.

2.3.2 The roofline model

The reason for this asymmetry is captured by the roofline model, a simple model of performance introduced for multicore processors [41]. A computation is characterised by its *arithmetic intensity*, the number of floating-point operations it performs for each byte it reads from memory. A processor has two limits: a peak rate of computation and a peak memory bandwidth. The attainable rate is the minimum of the peak computation and the product of bandwidth and intensity. A computation with low intensity is limited by the bandwidth, and one with high intensity by the computation; the intensity at which the two limits meet is called the ridge point [41].

Applied to a transformer, the model separates the two phases cleanly [42]. Each weight matrix of a linear layer must be read from memory once per forward pass. During prefill the same matrix multiplies the hidden states of hundreds or thousands of tokens, so each byte of weights is used for hundreds of operations: the intensity is high and the computation is the limit. During decoding, the matrix multiplies a single vector, so each weight is used for about two operations: the intensity is close to one and the bandwidth is the limit. For a 7-billion-parameter model in 16-bit precision, the linear layers reach intensities above one thousand operations per byte in prefill and about one in decode [42]. In datacenters this imbalance is softened: the decoding of many concurrent requests is grouped into batches, so that each weight read from memory serves to generate several tokens and the intensity rises [10]. A personal device serves a single user, one request at a time, and cannot rely on this remedy: this is where the asymmetry between the two phases weighs most.

Figure 2.3 draws the roofline of the thesis machine. Apple publishes its memory bandwidth, 120 GB/s, but no peak rate of computation [43]; the flat roof in the figure is therefore the rate actually reached during prefill, about 2.8 trillion operations per second, a lower bound of the peak. The ridge point lies at about 23 operations per byte. Generating a token sits at about two operations per byte, an order of magnitude below the ridge, whereas reading a document of hundreds of tokens sits an order of magnitude above it. The two phases of the same model, on the same machine, live in different regimes.

2.3.3 A lower bound on generation

Because decoding reads every weight once per generated token, its time per token cannot be lower than the size of the weights divided by the memory bandwidth, whatever the hardware. For the 3B model in 8-bit precision, whose weights occupy about 3.3 GB, this floor is 27 milliseconds per token on a machine with 120 GB/s; the measured 31.8 milliseconds are only about one sixth above it. The bound depends only on two numbers, the size of the model and the bandwidth of the

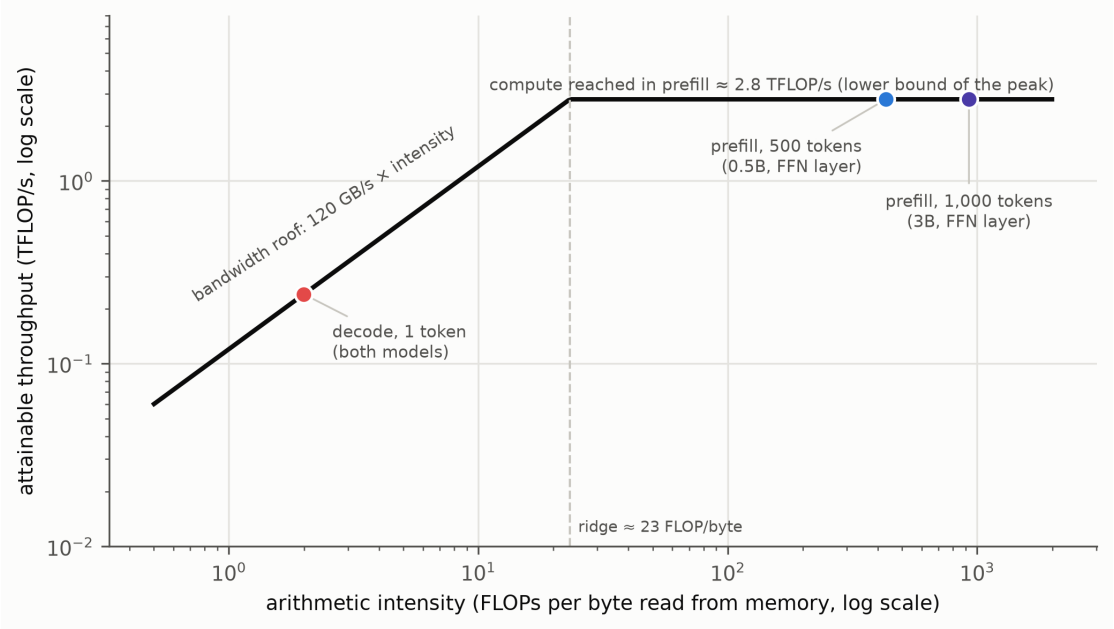


Figure 2.3: Roofline model of the thesis machine [41]. Sloped roof: Apple's memory bandwidth (120 GB/s). Flat roof: throughput reached in prefill, a lower bound of the unpublished peak. Intensities computed for one feed-forward layer of Qwen2.5 with 8-bit weights and 16-bit activations; attention is ignored.

machine, and it applies to every model that generates text. It is the key to comparing hardware in Section 2.4 and to the consequence drawn in Section 2.7.

Memory capacity sets a second, harder limit: the weights must fit. A model needs about two bytes per parameter in the 16-bit format used for training, one in 8 bits and half in 4 bits, plus small per-group scales (Figure 2.4). A 7-billion-parameter model in 16 bits needs 14 GB for its weights alone, beyond what most personal devices can spare [44]; in 4 bits it needs about 4 GB. The key-value cache, the activations, the operating system and every other model that runs at the same time come on top. A laptop with 16 GB can therefore hold a generator of 7–8 billion parameters in 4 bits next to a small verifier, but not much more.

2.4 Hardware for local inference

The cost model reduces the choice of hardware for local inference to three quantities. *Memory capacity* decides which models fit. *Memory bandwidth* decides how fast they can generate. *Computation* decides how fast they can read. Power and price complete the picture. This section surveys the hardware available in October 2026 along these axes, using the vendors' published specifications.

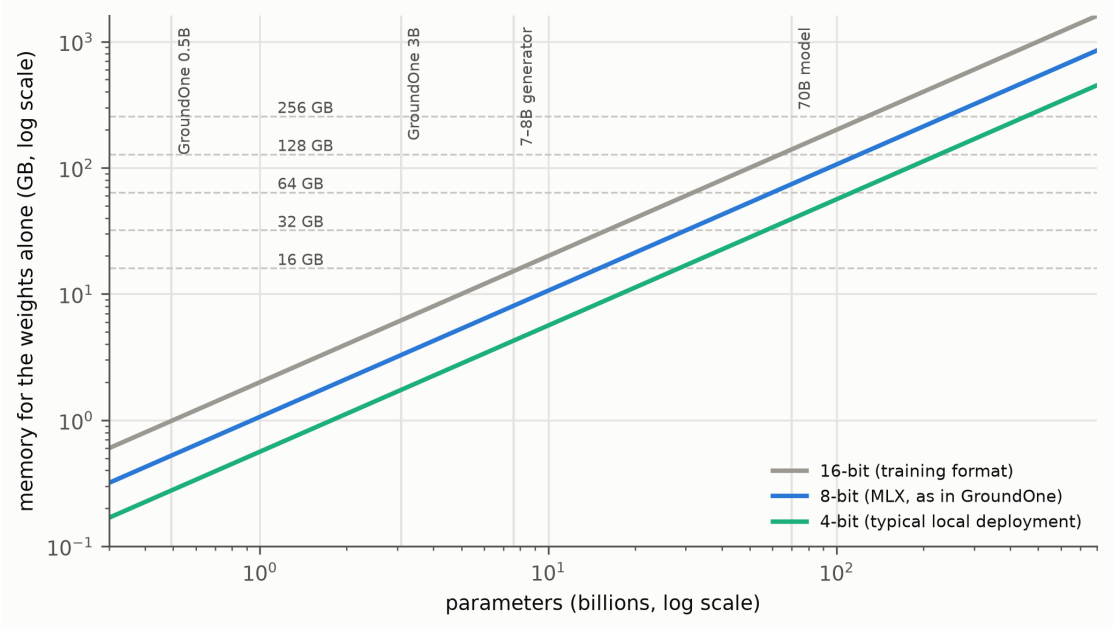


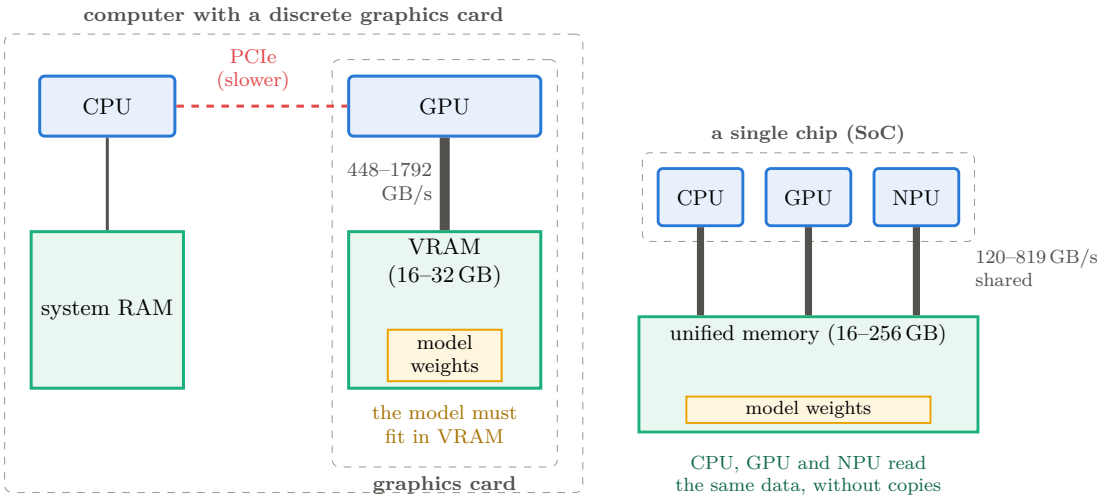
Figure 2.4: Memory needed for the weights of a model as a function of its size and precision (parameters times bits per weight, including half a bit of per-group scales for the quantised formats). Dashed lines: common memory sizes of local hardware. The key-value cache, activations and other running models come on top.

2.4.1 Classes of hardware

Two architectures dominate. In a *discrete* system, the graphics card has its own memory, fast but small, and the model must fit in it. In a *unified-memory* system, the processor and the graphics unit share one large pool of memory, so a model can use most of the machine's memory, at a lower bandwidth than a high-end graphics card. Apple's processors, NVIDIA's DGX Spark and AMD's Ryzen AI Max (Strix Halo) all follow the second design. Figure 2.5 compares the two architectures.

Table 2.2 and Figure 2.7 place the main options on the two memory axes. Figure 2.6 shows some of the systems discussed, to give a concrete idea of what they look like.

Laptops. Laptops with 16 to 32 GB and 120 to 150 GB/s, such as the MacBook Pro used in this thesis, are the most common personal computers on which local models run. They hold small models comfortably and models of 7–8 billion parameters in 4 bits. The thesis machine sits at the bottom left of Figure 2.7: if a verifier runs well there, it runs on everything above it.



(a) Discrete system: the GPU reads the weights from its own VRAM, very fast but limited to 16–32 GB; the rest of the system is reachable only through PCIe.

(b) Unified memory: CPU, GPU and NPU on the same chip share one pool of memory, larger but with lower bandwidth.

Figure 2.5: The two architectures of local inference hardware. Blue rectangles: compute units; green rectangles: memory; orange rectangle: the model weights; line width: indicative bandwidth. Capacity and bandwidth values from Table 2.2. In a discrete system a model that does not fit in VRAM must go through the much slower PCIe link; in a unified-memory system the model can occupy almost all of the machine's memory, but every generated token reads it at a lower speed than a high-end graphics card.

Desktop machines with unified memory. A second class, within or close to a budget of about three thousand dollars, offers much more memory at moderate bandwidth. AMD's Strix Halo processor, sold for instance in the Framework Desktop, lets the graphics unit use up to 96 GB of its 128 GB of memory; the 128 GB configuration was priced at 1,999 dollars at launch in August 2025, with a processor power of 55 W [49]. Apple's Mac mini with the M5 Pro reaches 64 GB at 307 GB/s [46]. Such machines can hold models of tens of billions of parameters or mixture-of-experts models, whose decoding reads only the active experts, at the price of a slower generation than a graphics card.

Dedicated AI machines. Above this budget, the Mac Studio with the M3 Ultra combines up to 256 GB with 819 GB/s [47], and NVIDIA's DGX Spark packs 128 GB of unified memory, a Blackwell graphics unit rated at one petaflop in 4-bit arithmetic with sparsity and 273 GB/s into a desktop box with a 240 W power



(a) 14-inch MacBook Pro with M4, the thesis machine.



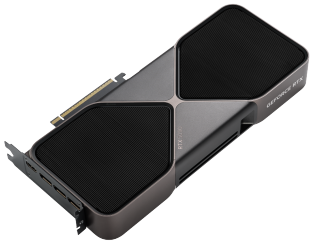
(b) Mac mini (2024, M4).



(c) Mac Studio.



(d) NVIDIA DGX Spark.



(e) GeForce RTX 5090 Founders Edition graphics card.

Figure 2.6: Some of the local inference systems compared in this section (images not to scale). Sources: (a), (c) Apple; (b) LoMit, Wikimedia Commons, CC0; (d), (e) NVIDIA. The vendors' images are promotional material, reproduced for the purpose of scientific documentation.

supply [48]. The two machines make opposite choices, and a direct comparison shows the cost model at work: on an 8-billion-parameter model with an 8,000-token prompt, the DGX Spark completes the prefill 3.8 times faster than the Mac Studio, thanks to its higher computation, while the Mac Studio generates 3.4 times faster, thanks to its three times higher bandwidth [51]. The same study combines the two machines, running the prefill on the Spark and the decoding on the Mac, which is the disaggregation of the two phases [40] brought to a desk.

Graphics cards. Discrete cards offer by far the highest bandwidth, up to 1,792 GB/s for the RTX 5090, but at most 32 GB of memory, at a power of several hundred watts [50]. They are the fastest option for any model that fits, and unusable for any model that does not.

Table 2.2: Hardware for local inference: vendors' published specifications (accessed 2026-10-11). Memory: range of the configurations offered; bandwidth: for those configurations; n/s: not stated by the source.

System	Memory (GB)	Bandwidth (GB/s)	Memory type	Source
MacBook Pro, M4 (thesis machine: 16 GB)	16–32	120	unified	[43]
MacBook Pro / Mac mini, M5	16–32	153	unified	[45, 46]
Mac mini, M5 Pro	24–64	307	unified	[46]
MacBook Pro, M5 Max	36–128	460–614	unified	[45]
Mac Studio, M4 Max	36–128	410–546	unified	[47]
Mac Studio, M3 Ultra	96–256	819	unified	[47]
NVIDIA DGX Spark (GB10)	128	273	unified	[48]
Framework Desktop, AMD Strix Halo	32–128 (96 to GPU)	n/s	unified	[49]
NVIDIA RTX 5090	32	1,792	discrete	[50]
NVIDIA RTX 5080	16	960	discrete	[50]
NVIDIA RTX 5060 Ti	16	448	discrete	[50]

Phones and mobile processors. At the other end, phones have 6 to 16 GB of memory and a bandwidth of a few tens of gigabytes per second shared by all processors [30, 52]. Their neural processing units are fast at 8-bit matrix multiplication and therefore at prefill: on a phone, a 7-billion-parameter model in 4 bits reads its prompt at about 770 tokens per second on the neural unit, against 37 on the graphics unit and 8 on the processor cores [52]. Research systems exploit this by running the prefill on the neural unit and the decoding elsewhere [53].

2.4.2 The asymmetry holds on every class

The two phases respond to different properties of the hardware, and recent hardware generations make the point explicit. Apple reports that its M5 processor, which

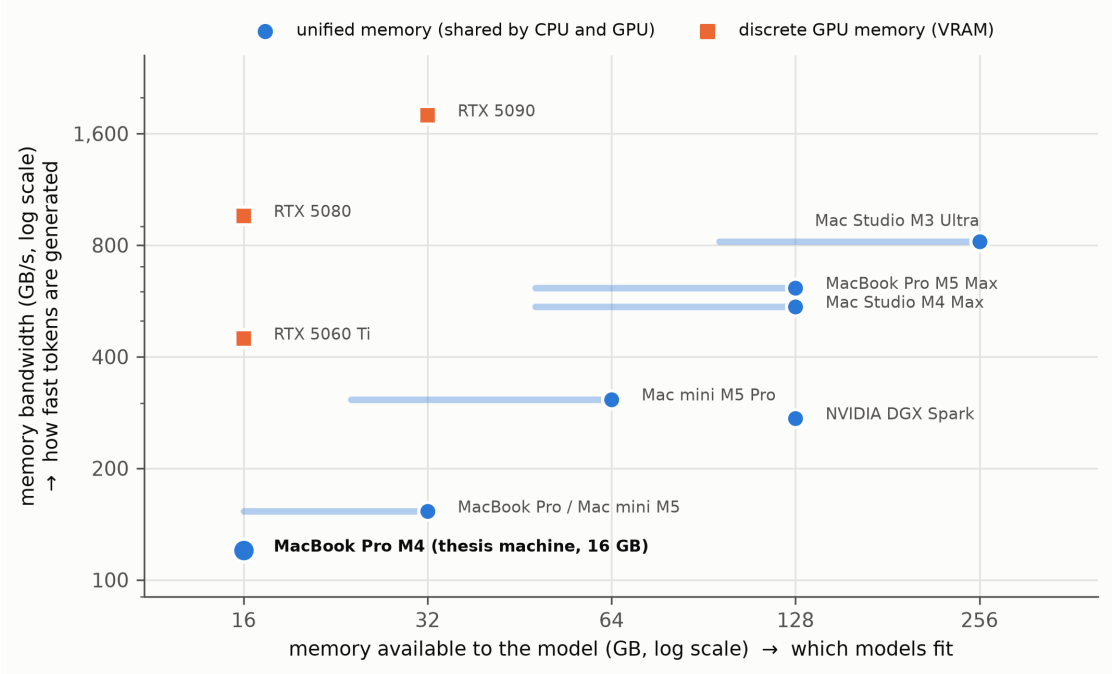


Figure 2.7: Local inference hardware on the two axes that decide what can run: memory capacity (which models fit) and memory bandwidth (how fast tokens are generated). Vendors' specifications: markers at the largest memory configuration, shaded bars over the range of memory sizes offered; the thesis machine is shown at its actual 16 GB.

adds dedicated matrix units to each graphics core, reduces the time to the first token, which is dominated by prefill, by 3.3 to 4.1 times with respect to the M4, while generation becomes only 1.19 to 1.27 times faster, in line with a memory bandwidth that grew from 120 to 153 GB/s [54]. Apple itself attributes the first figure to computation and the second to bandwidth. On edge devices, a study that compares 18 pairs of configurations finds that slowdowns accumulate in decoding, while the time to evaluate the prompt changes by at most 16 % [55]. On phones, for tasks whose output is short, prefill takes most of the latency [53]. Hardware is thus evolving to read faster, much more than to write faster.

Figure 2.8a applies the lower bound of Section 2.3.3 to the hardware of Table 2.2. On the fastest card, the 3B model could in principle generate a token in 1.8 milliseconds; on the thesis machine the floor is 27 milliseconds, and the measured time is close to it. Better hardware lowers the floor, but no hardware removes it.

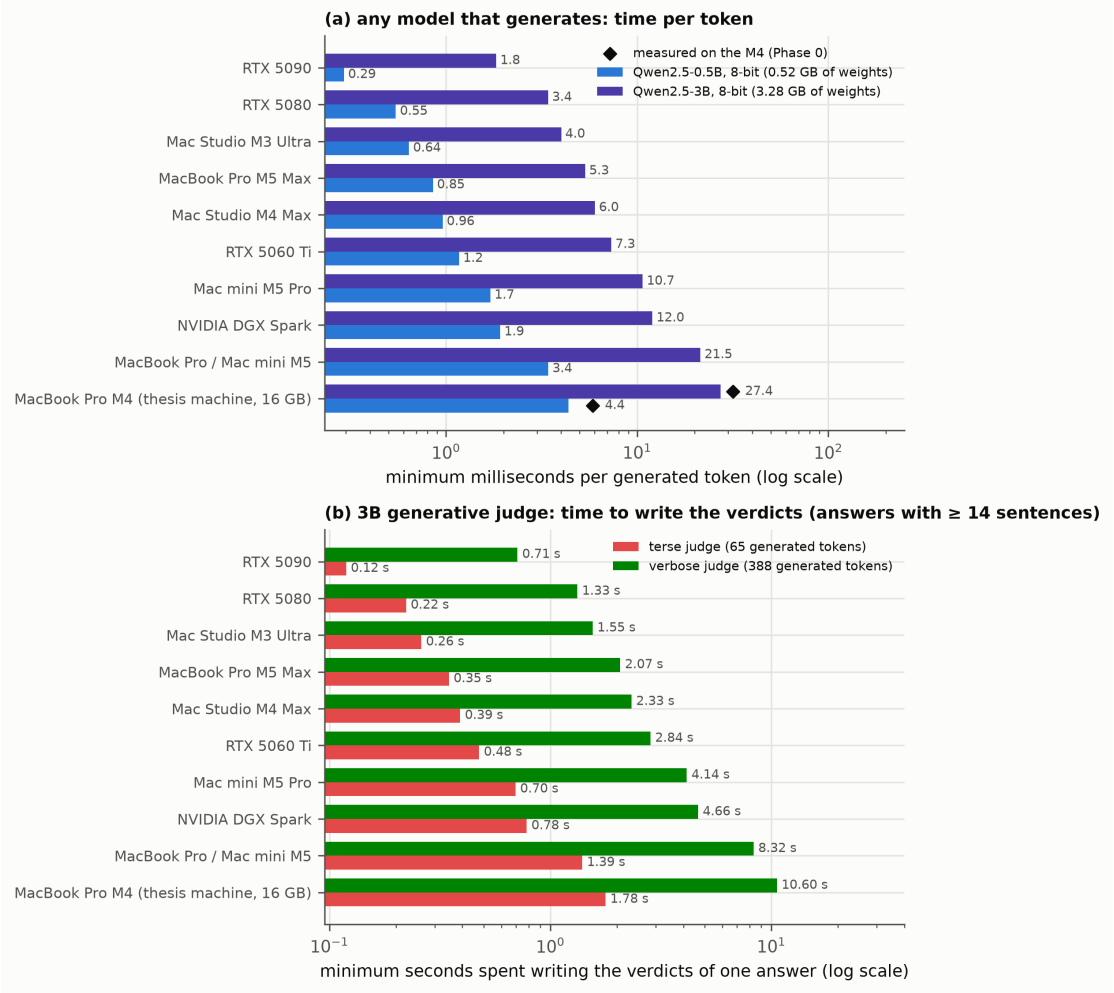


Figure 2.8: Lower bounds on the cost of generating text on each class of local hardware, computed as weight bytes divided by the vendor's bandwidth (one request at a time; key-value cache ignored). (a) Time per generated token with the two GroundOne base models in 8 bits; diamonds: decode time measured on the thesis machine. (b) Minimum time that a 3B generative judge spends only on writing the verdicts for an answer of fourteen or more sentences (median generated tokens of the latency protocol). A System One generates no token, so both costs are zero for it on every machine.

2.5 Runtimes and quantisation

2.5.1 Runtimes

The software for local inference has matured quickly. llama.cpp provides inference in C and C++ without dependencies, with integer quantisation and back ends for most consumer accelerators, and it defined the GGUF file format, which stores a quantised model together with its tokenizer and chat template in one file [2]. Distribution tools such as Ollama and LM Studio build on it and make downloading and serving a model a matter of one command. MLX is an array framework designed by Apple for the unified memory of its processors, with lazy evaluation and an interface close to NumPy [3]; its language-model package supports quantisation and LoRA fine-tuning directly on the device [56], and it is the runtime used throughout this thesis.

The runtime matters as much as the chip. A comparison of five runtimes on the same Apple machine reports decoding rates that differ by more than an order of magnitude, with MLX and MLC-LLM at the top and PyTorch's generic Metal back end at the bottom [57]; the study's numbers are not fully consistent with each other and should be read as indicative, but the spread is large enough to survive the uncertainty. On phones, an evaluation of several runtimes and devices finds large differences in speed, energy and thermal behaviour between runtimes and between devices [58].

2.5.2 Specialised inference engines

Next to the general runtimes, a second family of software appeared in 2026: inference engines written for a few models, which give up generality to push further the memory limits described in Section 2.3. DwarfStar (ds4), developed by Salvatore Sanfilippo, is a native engine optimised for a few large open-weight models, starting from DeepSeek V4 Flash, and aimed at Macs with at least 96 GB of memory, the DGX Spark and Strix Halo machines [59]. Two of its choices concern this thesis directly. The first is a key-value cache saved on disk: a conversation that has already been read can be resumed without repeating the prefill, which for models of this size is the most expensive part of a long request. The second is reading the weights directly from the SSD, which makes it possible to run models larger than the available memory, at the price of speed.

Colibri takes this second idea to the extreme [60]. It exploits the fact that a mixture-of-experts model uses only a small part of its weights for each token: it keeps the always-active part in memory and reads the experts from disk when the router requests them. In this way GLM-5.2, a model of 744 billion parameters, runs on a laptop with 25 GB of memory. The price is the one predicted by the

cost model: the bandwidth of an SSD is a few gigabytes per second, far below that of memory, and according to the authors generation drops to 0.05–0.1 tokens per second when cold on that laptop and to 1.83 tokens per second on a 128 GB Strix Halo machine [60]. It is the idea of LLM in a flash [44] applied to expert models: memory capacity stops being a hard limit and turns into a bandwidth limit, an even stricter one. Figure 2.9 illustrates the mechanism.

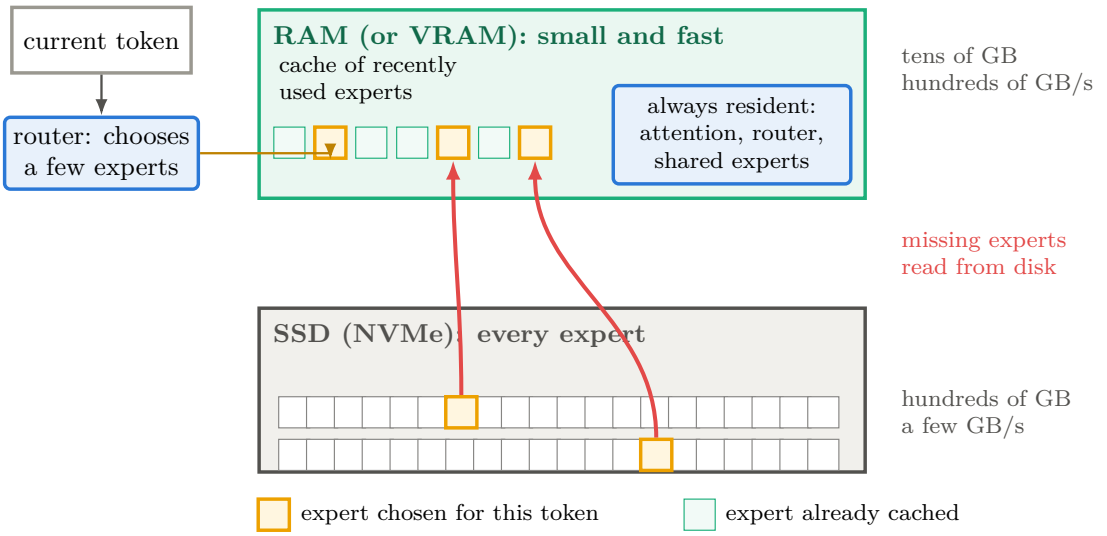


Figure 2.9: Streaming experts from the SSD in an engine such as Colibri. Memory holds the part of the model used by every token (attention, router, shared experts) and a cache of recently used experts; every expert is stored on the SSD. For each token the router chooses a few experts: those already cached are used at once, the missing ones are read from disk, at a bandwidth of a few gigabytes per second. For example, GLM-5.2 keeps about 10 GB resident and leaves about 429 GB of weights on disk [60]. The diagram is not to scale.

Colibri matters to this thesis for a second reason. It exposes the same System One interface as Jev, through which any model it runs answers closed questions by reading the probability of each option instead of generating the answer, and it includes three decision models natively: Laya, GLiNER2.5-Decide and Clef [60]. The authors report that, with Qwen3.6, four questions about the same document are answered 5.7 times faster than by generating the same answers on the same CPU. This is a figure reported by the project and not verified in this thesis, obtained with a different model and hardware, but it points in the same direction as the results of Chapter 7. That an engine designed to run huge models on modest hardware adopts the reading of decisions as a first-class mode confirms that the System One idea is spreading precisely where generation costs the most.

2.5.3 Quantisation

Quantisation stores the weights with fewer bits than the 16 used in training. On local hardware it serves two purposes at once: the model fits in less memory, and decoding becomes faster, because decoding time is dominated by reading the weights. Table 2.3 summarises the methods that shaped the field.

Table 2.3: Weight quantisation methods for language models.

Method	Idea and main result	Source
LLM.int8()	8-bit weights and activations; the few feature dimensions with large outliers are kept in 16 bits. No loss up to 175B parameters; saves memory but does not speed up small models	[61]
GPTQ	3–4-bit weights, quantised layer by layer with second-order information. 4-bit OPT-175B loses 0.03 perplexity; small models lose more; 3.2–4.5× faster decoding from bandwidth	[62]
AWQ	4-bit weights; the 1 % of channels that matter most, found from activation magnitudes, are protected by scaling. Robust to the calibration data; designed for on-device use	[63]
GGUF k-quants	2–8-bit block formats of llama.cpp, the de facto standard for consumer distribution	[2]
MLX affine	4- or 8-bit weights with a scale and a bias per group of 64; supports LoRA training on the quantised model (used in this thesis)	[56]
QLoRA	4-bit normal-float base model with 16-bit adapters, for fine-tuning on limited memory	[64]

Two results from this literature matter for this thesis. The first is that quantisation helps the bandwidth-bound phase. GPTQ attributes its speed-ups entirely to the matrix-vector products of decoding and explicitly obtains no reduction in computation [62]; AWQ illustrates the same point with a roofline: on a desktop graphics card, a 200-token prefill takes 10 milliseconds while generating 20 tokens takes 310 [63]. Reducing the bits of the weights therefore accelerates a model that generates far more than one that only reads. The second is that small models are more fragile. GPTQ loses much more accuracy at 4 bits on a 1.3-billion-parameter model than on a 175-billion one, and models of three billion parameters or fewer become unusable at 3 bits on phones [62, 58]. LLM.int8() shows that a handful of outlier features control a large share of the attention probability mass [61]. For a model whose output is a probability that must be calibrated, rather than a text that must merely be fluent, these observations argue for a conservative precision: GroundOne uses 8-bit weights.

2.5.4 Faster generation without fewer tokens

A different family of methods accelerates generation without changing the model. In speculative decoding, a small draft model proposes several tokens, and the large model checks all of them in a single forward pass, accepting the longest prefix that it would have produced itself; the output is identical to that of the large model, and the speed-up reaches two to three times [65]. The method is useful for any system that must generate, but it does not change the nature of the cost: every accepted token still requires a forward pass of the large model, and the total amount of computation grows [65]. It makes writing cheaper; it does not make it free.

2.6 Energy

On a battery-powered device, energy is a property of the method, not only of the hardware. Measurements on datacenter hardware show that the energy of a request grows with the number of generated tokens and depends strongly on model size and task [66, 67]. On phones, a 3-billion-parameter model in 4 bits spends between 0.58 and 0.76 joules per generated token, with sustained power around 14 W and surface temperatures close to 48 °C [58]; a rule of thumb from work on small models puts the cost at about 0.1 joules per generated token for each billion parameters [37]. Moving the prefill to a phone's neural unit reduces its energy by 1.9 to 4.3 times with respect to the graphics unit [53].

The distinction between the two phases also applies to power. Prefill keeps the arithmetic units busy and draws more power; decoding waits for memory and draws less, but for much longer [40]. A method that saves time therefore saves energy too, but by a smaller factor, and Chapter 7 measures exactly this effect: GroundOne spends 5.5 joules per long answer with the 0.5B model and 33 with the 3B, less than any generative judge of the same size, but with an advantage smaller than in time (Section 7.10).

2.7 Consequences for verification on the device

The chapter can now be summarised in one observation: on a local device, which serves one request at a time, reading a token costs far less than writing one, fourteen to sixteen times less on the thesis machine (Sections 2.3.1–2.3.3), and recent hardware generations accelerate reading much more than writing (Section 2.4.2). This has a direct consequence for a verifier that runs next to the generator.

A generative judge verifies an answer by reading the document and the answer and then writing its verdicts. It pays a prefill, like any method that reads the document, and then one decoding step for every token it writes. In the latency

protocol of this thesis (Section 7.9), a terse 3B judge writes a median of 65 tokens for an answer of fourteen or more sentences, and a verbose one, which explains each verdict, writes 388. The lower bound of Section 2.3.3 turns these counts into a minimum time that no implementation can beat (Figure 2.8b): on the thesis machine, at least 1.8 seconds for the terse judge and 10.6 for the verbose one, on top of the prefill; even on the fastest graphics card, 0.12 and 0.71 seconds.

A System One removes this term altogether. It reads the document and the answer once and takes every decision from the logits computed during that single reading, so its cost is the prefill alone: the compute-bound phase, the one that local hardware performs well and that new hardware accelerates most. Its advantage over a judge is therefore not a property of one machine but of the cost structure of local inference, and it is largest exactly where inference is local. Locality, which at first sight is only a constraint on the size of the model, becomes the reason why the design matters.

Three requirements for the verifier follow from this chapter, and they guide the method of Chapter 4. It must generate no tokens, so that its cost is a single prefill. It must use a model small enough to share a 16 GB machine with a generator and a retriever, quantised conservatively because its output is a probability. And it must read its decisions from logits, which only a local model exposes, so that they can be calibrated.

Chapter 3

Background and Related Work

This chapter reviews the work on which GroundOne builds and with which it is compared. The cost of inference on a local device, which motivates the design, was treated in Chapter 2. The chapter starts from System One models, the idea of reading decisions from a language model instead of generating them, and the rapidly growing ecosystem of decision models. It then reviews the problem of hallucination in retrieval-augmented generation and the methods that verify answers against their sources, and closes with calibration, parameter-efficient fine-tuning and the gap that this thesis addresses.

3.1 System One models

The term *System One model* was introduced by TypeSafe with Jev, a hosted model that answers closed questions about a given state with typed outputs, such as a boolean, a choice among options or a score, without generating text [14]. The name evokes the fast, intuitive mode of thinking described by Kahneman, as opposed to deliberate reasoning. The vendor reports latencies of tens to hundreds of milliseconds and large cost savings with respect to generative judges, but these figures are claims, and the architecture is not disclosed.

Open implementations followed, and they differ in how the probabilities are produced. The most common family keeps the language-model head and reads the logits of the option letters at the answer position, after a prompt that presents the question as a multiple choice. SemIf applies this readout to a frozen 4B model and reports that 21 criteria about one document are answered in about one second against more than five seconds for the best generative baseline that produced valid JSON [15]. Rizzo-Flow fine-tunes a 4B model with LoRA on typed decisions,

serves the same interface as Jev and reports gains in accuracy and calibration after fine-tuning [16]; it is the general-purpose System One used as a baseline in this thesis. A second family encodes the state and each candidate answer separately and compares their embeddings: Contrastive Language Models are very fast when candidates are many and reusable, but in their own comparison they agree with Jev's decisions much less than a model that lets state and candidate interact [68]. A third family uses small encoders with a dedicated decision head. Laya, for example, scores each option at its own mask token of a ModernBERT encoder of about 400M parameters; the questions of a request are scored in one batched call, but each question is encoded with its own copy of the context [69, 70].

The evidence for these systems is uneven. Most of it comes from their authors, often on a single machine and a single benchmark. Independent evaluations are recent and less favourable to the small open models: an evaluation of Jev on 37 datasets finds its choice probabilities well calibrated, but its binary probabilities, while ranking well, poorly placed with respect to a fixed threshold of 0.5, and reports a mean balanced accuracy of 78.6 on LLM-AggreFact [20]; an independent reproduction of Laya finds a different calibration from the one claimed and a strong dependence on a temperature fitted to the target data [21]; contrastive benchmarks with clear protocols place small open models far behind Jev [71]; and typed decision models have been shown to follow the label of an option rather than its definition when the two conflict [70]. On 9 October 2026 the Hugging Face hub officially introduced a tag for these *decision models* [17], which the following day already grouped about 1,500 models from some 600 authors, of very different quality. In the rest of the thesis the two terms are used as synonyms: *System One* is the name introduced by TypeSafe with Jev, *decision model* the one later adopted by the ecosystem.

Within weeks of Jev's release, established organisations published open decision models of their own. Cloudflare's Clef adds a dedicated decision head to a decoder and answers all questions in a single pass over the state, the design that had only been inferred for Jev [19]; Liquid AI's d1-3B reads restricted answer letters from a 3B model whose state is shared by all questions [18]; Perplexity fine-tunes a 27B model with a letter head [72]. A community leaderboard, the Decision Index, scores these models on a common suite of public benchmarks that includes RAGTruth at the level of whole answers [22]. Its results draw a clear line: general-purpose decision models with fewer than one billion parameters barely exceed the trivial baseline on RAGTruth, and only models of tens of billions of parameters detect hallucinated answers reliably. Studies of Jev itself add that its probabilities are calibrated when pooled over many tasks but not on each task separately, that thresholds and temperatures must be fitted on the target workload [73, 74], and that choosing a threshold through a lower confidence bound makes a decision-only judge safer to deploy [74]. Runtimes followed: llama.cpp now serves the System One

interface natively [2]. None of these models, however, is specialised in groundedness, and none reports sentence-level, three-way verification on the benchmarks used in this thesis.

The letter readout itself has been studied outside the System One literature. In multiple-choice evaluation, the probability of the first answer token and the answer that an instruction-tuned model actually writes often disagree [75], and the readout is biased towards particular option identifiers, a bias that can be estimated and removed [76]. Both findings concern zero-shot models and suggest that a letter readout used as a decision needs to be trained, calibrated, or both. No open System One has been specialised in groundedness verification and compared, under controlled conditions and on the same hardware, with generative judges, specialised verifiers and general-purpose System One models, one of them of the same size.

3.2 Retrieval-augmented generation and hallucination

Retrieval-augmented generation gives a language model access to documents it was not trained on: a retriever selects passages relevant to the question, and the model writes its answer with these passages in its context [1]. The approach is the standard way to build assistants over private documents, and it can run entirely on a local machine. It does not, however, remove hallucination. Surveys distinguish errors of *factuality*, with respect to world knowledge, from errors of *faithfulness*, with respect to the given input or context [9]; a RAG answer can be unfaithful to the very passages it was given, by contradicting them or by adding information they do not contain. RAGTruth documents how frequent this is: it collects about 18,000 answers written by six language models in three retrieval-augmented tasks and annotates, at the level of character spans, the parts that conflict with the context or are not supported by it [8]. Its distinction between conflicting and baseless content corresponds to the classes *contradicted* and *not mentioned* used in this thesis.

3.3 Groundedness verification

Verifying an answer against its source is closely related to natural language inference, the task of deciding whether a premise entails, contradicts or is neutral towards a hypothesis. Large inference datasets such as MultiNLI [77] and the adversarial ANLI [78] provide training data, and VitaminC [79] adds contrastive pairs in which a minimal revision of a Wikipedia sentence flips the verdict, forcing a model to look at the exact fact rather than at word overlap. Applying these models to long generated texts requires decomposition: SummaC splits both document

and summary into sentences and aggregates sentence-level inference scores [80], and FActScore decomposes long generations into atomic facts and verifies each of them [81]. Sentence-level verification, as in GroundOne, is a middle ground: sentences are natural units of an answer, a user can act on a flagged sentence, and the number of decisions grows with the length of the answer.

Two families of verifiers dominate practice. Specialised encoders are small and fast. AlignScore trains a single alignment function on a large mixture of tasks [11]; MiniCheck trains small checkers on synthetic data designed to require reasoning over several sentences and matches much larger models on grounding benchmarks [12]; HHEM is an openly released hallucination evaluation model of the same kind [13]. These models score each pair of document and sentence independently and return a single probability of support. The second family uses a language model as a judge [82]. RAG evaluation frameworks such as RAGAS [83] and ARES [84] rely on judges, Lynx fine-tunes a judge specifically for hallucination detection [85], and Self-RAG trains the generator itself to emit critique tokens [86]. Judges are flexible, but their cost grows with what they write, small models often break the required output format, and grammar-constrained decoding is needed to guarantee a parsable answer [87]. LLM-AggreFact collects eleven human-labelled grounding datasets under a common binary protocol and is the reference benchmark for both families [12, 88].

3.4 Calibration and selective prediction

A probability is calibrated when, among the predictions made with a given confidence, the share of correct ones matches that confidence. Modern neural networks are often miscalibrated, and a single temperature applied to the logits, fitted on held-out data, is a simple and effective correction [89]; the expected calibration error, computed over bins of confidence, is the usual measure [90]. For prompted language models, a per-class bias can also be needed to correct a systematic preference for some answers [91]. Language models know to some extent what they know: their token probabilities on closed questions are often informative about correctness [92]. When the probabilities are not accessible, the confidence can be asked for in words; for large models fine-tuned with human feedback this verbalised confidence can be better calibrated than the probabilities estimated from samples [93], but systematic evaluations find verbalised confidence coarse and overconfident, with values concentrated on a few round numbers [94]. A calibrated probability becomes useful through selective prediction: a system acts on its confident decisions and defers the others, and the risk–coverage curve shows how the error rate falls as more decisions are deferred [95].

3.5 Parameter-efficient fine-tuning

Specialising a pretrained model does not require updating all its weights. Low-rank adaptation freezes the model and trains, for selected weight matrices, a low-rank update that can be merged into the weights or kept as a small separate adapter [24]; applying it to all linear layers matters more than a high rank, and it combines with quantised base weights to fit large models in little memory [64]. The Qwen2.5 family provides open models from 0.5B parameters upwards with a shared tokenizer, which makes it possible to compare model sizes with the same recipe [23]. Fine-tuning small datasets is, however, known to be unstable: results vary with the random seed through both the initialisation of new parameters and the order of the training data [96]; some runs fail entirely, a behaviour linked to optimisation details such as the absence of bias correction in Adam [97]; and clipping the norm of the gradient is the standard defence against occasional exploding updates [98].

3.6 Research gap

The pieces needed for a local, fast and calibrated verifier exist separately: compact open models and runtimes that run them on a laptop, the System One idea of reading decisions instead of generating them, specialised verifiers, and post-hoc calibration. What is missing is an independent and controlled study that puts them together for one task. In particular, no work has measured whether a compact System One specialised in groundedness verification can match generative judges and dedicated encoders on that task, where its advantage comes from, among specialisation, the readout and the model size, whether its probabilities are good enough to set thresholds and defer decisions, and how its time and energy compare with those of a judge on the same consumer hardware. The rest of this thesis addresses these questions.

Chapter 4

GroundOne: Method and Implementation

GroundOne is a compact language model used as a System One: it reads a document and an answer once and returns a verdict for every scored sentence of the answer by reading the logits of a few answer letters, without generating text. This chapter describes the decision it makes, the prompt that carries all the decisions of one answer in a single forward pass, the training objective, the calibration of the output probabilities, the rule that turns probabilities into served labels, and the software that implements them.

4.1 Task and decision

For every sentence of an answer GroundOne makes one decision among three classes, defined in Section 5.1: the sentence is *supported*, *contradicted* or *not mentioned* with respect to the document. The three classes are mapped to the letters A, B and C, and the model outputs a probability for each of them. A binary groundedness score and the strict and lenient policies are not separate questions to the model; they are computed from these three probabilities, so that a single forward pass serves every use. Asking for the policies directly would add nothing: each of them is a deterministic function of the three-way class, so separate questions would multiply the decisions without adding information, and could give the model inconsistent targets for the same sentence.

4.2 Reading all verdicts in one forward pass

The prompt contains a short instruction, the document and the answer. Every sentence of the answer is numbered, and every sentence that carries a decision is followed by a marker, the string “ `Verdict:`”. Sentences without checkable content (Section 5.3) are kept as context but receive no marker. The model processes the whole sequence once, and at the last token of each marker the logits of the letters A, B and C are read and normalised with a softmax restricted to these three letters. Because a decoder only attends to earlier positions, the verdict on sentence i depends on the document and on sentences $1, \ldots, i$, which is what verification needs: references such as pronouns point backwards. Nothing after the last marker affects any verdict, so the sequence ends there.

Two details make the readout efficient and exact. First, the prompt is built by tokenising its pieces separately and concatenating them, so the position of every marker is known exactly and is identical at training and inference time. Second, the output projection onto the vocabulary is applied only at the marker positions. The transformer body runs over the full sequence, but the large vocabulary matrix is multiplied only by the few hidden states that are read.

This design determines the cost of a verification. If S is the length of the document and answer, K the number of decisions and m the length of a marker in tokens, the System One costs one prefill of $S + Km$ tokens. A generative judge must also decode its verdicts: with n tokens per verdict and a per-token decoding time t_{dec}, its cost is roughly $\text{prefill}(S) + K\,n\,t_{\text{dec}}$. Since m is three tokens and decoding is much slower per token than prefill on the target hardware, the gap widens as K grows. On the M4 used in this thesis, prefill costs about 0.37 ms per token for the 0.5B model and 2.2 ms for the 3B, whereas decoding costs 5.9 and 31.8 ms per generated token: one decoded token is as expensive as fourteen to sixteen prefilled ones. The two phases also stress the hardware differently. Prefill processes many tokens at once and is limited by computation, while decoding produces one token at a time and is limited by the bandwidth needed to read the weights [10, 40]. This difference returns in the energy measurements of Chapter 7.

A second design reads the shared prefix once and then appends one short question per sentence, scored in a batch. It needs a key-value cache replicated K times, whereas the marker design needs a single pass. It is kept as a fallback and is the natural format for an untrained model, which understands an explicit question but has no reason to treat a marker as one; for this reason it is used by the untrained baseline (Section 6.2).

A classifier, not a faster generator. The readout restricts the decision to three letters, and this restriction matters. After the marker, the fine-tuned model would not write the letter that is read: its most probable next token is the same letter

preceded by a space, and the three readout letters together hold only about 4–5 % of the probability mass at the markers. Training with a loss restricted to these letters shapes their relative logits, not their absolute probability, so the readout behaves as a three-way classifier placed on top of the language model. This is what defines a System One: the answer is never decoded, it is chosen among the admissible options. It also echoes the observation that the probability of the first answer token and the answer a model actually writes can disagree [75], and that letter choices carry selection biases [76], which a specialised and calibrated readout must absorb. A control experiment that takes, with the same weights and the same pass, the token that generation would emit instead of the restricted readout quantifies the difference (Section 7.6).

4.3 Training objective

GroundOne is obtained from Qwen2.5-0.5B-Instruct and Qwen2.5-3B-Instruct [23] with low-rank adaptation (LoRA) [24]. Adapters of rank 16, with scaling factor 2 and dropout 0.05, are attached to all linear layers of all transformer blocks, since covering every layer matters more than a high rank [64]. The base weights stay frozen in 8-bit precision. One training example is one answer with all its scored sentences, and the loss is computed at every marker of the answer.

The loss is a cross-entropy over the three letters only, so the prompt itself is never supervised and the training signal is entirely the verdicts. Because about 88 % of the training sentences are supported, the classes are weighted so that each contributes equally to the loss: with the training distribution this gives weights of about 0.38 for supported, 7.5 for contradicted and 4.4 for not-mentioned sentences. Without such weights a model can reach a low loss by answering “supported” almost always. A useful consequence is that a model which learned only the class frequencies would stay at a loss of $\ln 3 \approx 1.10$, so a loss below this value shows that the text is being used.

Optimisation uses AdamW without weight decay and, as in the default configuration of MLX, without bias correction of the moment estimates; gradients are not clipped. The learning rate of 10^{-4} is reached after a linear warm-up over the first 3 % of the steps and then decayed to zero with a cosine schedule, over one epoch of the training answers. Answers of similar length are grouped into micro-batches under a budget of padded tokens, and gradients are accumulated so that each optimiser step sees about 16 answers, or about 110 decisions; this keeps a few contradicted sentences in every step despite their rarity. Gradient checkpointing keeps memory within the 16 GB of the laptop. The model is evaluated on the development set at regular intervals, and the checkpoint with the highest three-way balanced accuracy, averaged over the three tasks, is kept. Section 6.1 lists the

settings of each run.

4.4 Calibration

The class weights train the model as if the three classes were equally frequent, so its raw probabilities are not calibrated to the real frequency of errors. A post-hoc map is therefore fitted on the development set: the three logits are divided by a temperature T and shifted by a bias per class, $\mathrm{softmax}(z/T + b)$ with the bias of the supported class fixed at zero, and T and b minimise the negative log-likelihood of the development labels [89]. The bias plays the role of the contextual correction proposed for prompted language models [91], fitted here on labelled data. A temperature alone changes the probabilities but neither the most probable class nor the ranking of the sentences. The bias, however, can change which class is most probable, and with three classes it can also reorder $P(\text{supported})$ slightly between sentences. For this reason the classification metrics and the AUROC are computed on the raw output, and the calibrated probabilities are used where a probability must mean a frequency: for the calibration metrics, for thresholds and for selective prediction (Section 7.7).

Calibration is measured mainly with the expected calibration error (ECE) [90, 89]. The predictions are divided into M bins of equal width according to their confidence, with $M = 15$ in this thesis. For each bin B_m the mean confidence $\mathrm{conf}(B_m)$ is compared with the observed frequency $\mathrm{acc}(B_m)$, and the gaps are averaged with weights proportional to the size of the bins:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \Big|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\Big|. \tag{4.1}$$

In the binary variant the confidence is $P(\text{supported})$ and the observed frequency is the share of supported sentences in the bin; in the top-label variant for three classes the confidence is the highest probability and the observed frequency is the share of correct predictions. ECE does not compare the class distribution of the predictions with that of the test set; it asks whether, among the sentences to which the model assigns a given confidence, that confidence matches reality. It depends on the choice of bins, and a constant predictor that always outputs the base rate has an ECE close to zero while discriminating nothing. For these reasons ECE is always reported together with the Brier score, the negative log-likelihood and reliability diagrams, and the ability of the probabilities to separate correct from incorrect decisions is measured separately, with the AUROC and the area under the risk–coverage curve.

From probabilities to served labels. A user of the verifier sees a label and a probability for each sentence, and the two must agree. The raw decision, the most

probable class of the uncalibrated output, is the one evaluated in this thesis, but it can disagree with the calibrated probability shown next to it: a sentence may be labelled "not mentioned" while its calibrated $P(\text{supported})$ is slightly above one half. The served output therefore derives every element from the calibrated probabilities with a single rule. A sentence is labelled supported if and only if $P(\text{supported}) \geq \tau$, and otherwise receives the more probable of the two remaining classes; the strict policy flags every sentence that is not supported, and the lenient policy flags only the contradicted ones. The threshold τ is fitted on the development set to maximise the three-way balanced accuracy, and a user can change it at request time to choose another operating point without retraining.

4.5 Software

GroundOne is implemented as an installable Python package on MLX [3, 56], the array framework for Apple silicon, and every training and inference step described above runs in it on the same laptop. The package contains the dataset builder and its checks, the training loop, the calibration, the evaluation on the frozen test sets, the baselines and a memory guard that runs long jobs one at a time and stops a job before the system runs out of memory. A command-line interface and an HTTP server expose the verifier; the server offers a native endpoint and an endpoint compatible with the interface used by other System One models, so that existing clients can use GroundOne by changing the address. The native endpoint receives a document and an answer, segments the answer with the same rule used to build the data, runs the single forward pass of Section 4.2 and returns, for every sentence, whether it was scored, the three calibrated probabilities, the served label, the strict and lenient flags, and the decision rule and threshold that produced them; it also reports the number of input tokens, the number of generated tokens, which is always zero, and the time spent. The compatible endpoint answers generic closed questions over a shared context with the question-per-sentence format. All model computation runs on a single long-lived worker thread, because the Metal context of the GPU must not move between threads, and nothing leaves the machine: models, data and requests stay local.

Chapter 5

Data

GroundOne learns a single decision, and the data must teach exactly that decision and measure it reliably. This chapter first defines the decision and its labels, then describes the three public datasets that provide them and the way they are turned into training, development and test sets. The last sections report how the reliability of the test sets was checked without new annotation, what an exploratory analysis of the result reveals, and how leakage between training and test data was ruled out.

5.1 Task and labels

The input is a retrieved document and an answer written by a language model, split into sentences. For every sentence the model decides whether, *with respect to the document*, the sentence is *supported*, *contradicted* or *not mentioned*. A sentence is supported when everything it states is said or directly implied by the document. It is contradicted when some part of it conflicts with the document, and not mentioned when it adds information that the document neither contains nor contradicts. A sentence that contains both a conflicting and an unverifiable part is labelled as contradicted, because a contradiction is the more serious error. Only the document counts: a statement that happens to be true in the world but is absent from the document is still not mentioned.

The number of decisions for one answer is the number of its scored sentences, denoted K. This quantity is central to the thesis. A System One reads the document and the answer once, so each additional decision is almost free, while a generative judge has to produce text for every sentence. The comparison with generative judges is therefore made as a function of K, and the data must contain long answers as well as short ones.

The model outputs three probabilities per sentence, and every other quantity is derived from them rather than learned separately. The binary groundedness

score, used for comparison with published binary benchmarks, is $P(\text{supported})$. A *strict* policy, under which anything not stated in the document is an error, flags a sentence when $P(\text{supported})$ falls below a threshold τ. A *lenient* policy, under which only contradictions are errors, flags it only when it is judged contradicted. The user can thus choose the notion of error, and the threshold, at inference time without retraining the model; Section 4.4 gives the exact rule used by the served verifier.

5.2 Source datasets

The dataset is deliberately built from as few sources as possible: one for training and three for testing (Table 5.1). Fewer sources mean a single definition of the labels, a single text format and fewer places where the construction can go wrong.

Table 5.1: Source datasets and their role in GroundOne.

Dataset	Licence	Role	Labels
RAGTruth [8]	MIT	train, dev, test T1	character spans of four hallucination types on LLM answers
LLM-AggreFact [12, 88]	CC BY-ND 4.0	test T2 only	binary (supported / unsupported) per claim
VitaminC [79]	CC BY-SA 3.0	test T3 only	supports / refutes / not enough information

RAGTruth. RAGTruth [8] contains 17,790 answers that six language models (GPT-3.5, GPT-4, Llama-2 7B, 13B and 70B, and Mistral-7B) wrote in three retrieval-augmented tasks. In question answering the context is a set of passages retrieved for an MS MARCO question [99]. In summarisation it is a news article from CNN/DailyMail [100, 101] or from recent news, and in data-to-text generation it is a Yelp business record in JSON [102]. Two trained annotators marked the character spans of each answer that the context does not support, with a third review in case of disagreement, and assigned each span one of four types: evident or subtle *conflict*, and evident or subtle *baseless information*. The paper reports an agreement of 91.8 % at the level of answers and 78.8 % at the level of spans. It is the only public dataset that combines natural multi-sentence answers from several models with a distinction between contradicted and unsupported content, which is

why it is the only training source. Its official split separates 450 test sources from the remaining training sources.

LLM-AggreFact. LLM-AggreFact [12, 88] aggregates eleven human-labelled fact-checking datasets of (document, claim) pairs. They cover summarisation (AggreFact-CNN and XSum [103], TofuEval [104]), retrieval-augmented and long-form question answering (RAGTruth [8], ClaimVerify [105], ExpertQA [106], LFQA [107]) and fact checking (WiCE [108], REVEAL [109], FactCheck-GPT [110]). Its public leaderboard makes results comparable with published systems. The licence states that the data must not be used to train models [111], so in this thesis it is used only for evaluation: nothing is trained, calibrated or selected on any of its splits.

VitaminC. VitaminC [79, 112] is a fact-verification dataset built from Wikipedia revisions that change a fact. For each revision, trained annotators wrote claims that one version of the sentence supports and the other refutes; when no contradiction could be written, the pair was labelled as having not enough information. Because the two versions of the evidence are almost identical, a model must look at the exact fact rather than at word overlap. On a re-annotated sample of 2,000 pairs the authors report a Fleiss' κ of 0.71. VitaminC plays a specific role in this thesis. RAGTruth publishes no agreement for the hallucination *type*, so it cannot alone show that the boundary between contradicted and not mentioned is learned. VitaminC provides that boundary with labels of known reliability, in a different domain.

Several other candidates were examined and not adopted. Natural language inference corpora such as ANLI [78] have short premises far from retrieval-augmented contexts, and VitaminC covers the same three-way distinction with factual text. Synthetic or binary-only training data, such as the data released with MiniCheck [12], does not carry the three-way labels. Datasets whose labels were generated by a language model, or which reuse RAGTruth test answers, were excluded to keep the test sets independent and the labels human.

5.3 Construction

Sentence segmentation. Answers are first split at line breaks, because they often contain lists and headings, and each line is then segmented with PySBD [113], a rule-based segmenter that handles abbreviations and decimal numbers. Leading list markers such as "3." or "*" are removed, since they are layout rather than content. The same segmenter is used for training, evaluation and inference, so the model always sees sentences cut in the same way.

Sentences without checkable content. Some sentences make no claim about the document: greetings, headings that introduce a list, citation-only lines such as "(Passage 2)" and refusal formulas. They remain in the numbered answer as context but are not decisions, and they carry no label. A short rule detects them, and it applies to 8.2 % of all RAGTruth sentences, mostly list headings. The rule concerns only formatting, such as list markers, citation-only lines and conversational formulas, and it never changes a label; its coverage was checked against the independent segmentation of LLM-AggreFact described in Section 5.5.

Labels. A scored sentence is labelled as contradicted if it overlaps a conflict span, as not mentioned if it overlaps only baseless-information spans, and as supported otherwise. Only 27 of the 14,226 annotated spans (0.19 %) touch no scored sentence, so almost no annotation is lost in the conversion. For LLM-AggreFact the binary label is kept as released. VitaminC labels are mapped directly: *supports* to supported, *refutes* to contradicted and *not enough information* to not mentioned.

Exclusions and splits. Answers flagged by RAGTruth as incorrect refusals or truncated are removed, as are answers without any scored sentence and answers whose document and answer together exceed 2,048 tokens, the length limit imposed by memory on the 16 GB laptop. The official RAGTruth test split becomes test set T1. The training sources are shuffled with a fixed seed: 10 % of them form the development set and the rest the training set, so that answers about the same source never appear in both. Six training answers about one news article that also appears in the LLM-AggreFact test set are removed. From the real part of the VitaminC test split, which is unbalanced, 1,000 pairs per class are sampled with a fixed seed to form T3.

5.4 The resulting dataset

Table 5.2 summarises the five files. The training set contains 13,203 answers and 90,795 labelled sentences. Records are stored per answer, with one label per sentence, so the same data can feed different ways of reading the K decisions of an answer.

RAGTruth is strongly imbalanced: about 88 % of the labelled sentences are supported, and contradictions are only 3 to 4 % (Figure 5.1). This is the natural rate of errors in model answers, and it is kept: the test set reflects what a verifier meets in practice, while the imbalance is handled during training. T3 compensates with a balanced three-way test. Answers have 6.9 scored sentences on average in training and 6.5 in T1, with a maximum of 28 (Figure 5.2). For the analysis by length, T1 is divided into four bands of K: 756 answers with 1–4 sentences,

Table 5.2: The GroundOne dataset. A decision is a labelled sentence; T2 is binary.

Split	Source	Records	Decisions	Supported	Contradicted	Not mentioned
Train	RAGTruth	13,203	90,795	79,898	4,039	6,858
Dev	RAGTruth	1,468	9,996	8,827	410	759
T1	RAGTruth test	2,647	17,284	15,745	584	955
T2	LLM-AggreFact test	29,320	29,320	22,832	6,488 unsupported	
T3	VitaminC test	3,000	3,000	1,000	1,000	1,000

1,288 with 5–8, 497 with 9–13 and 106 with 14 or more. Cells with fewer than 30 sentences of a minority class, such as contradicted sentences in question answering, are reported but marked as indicative.

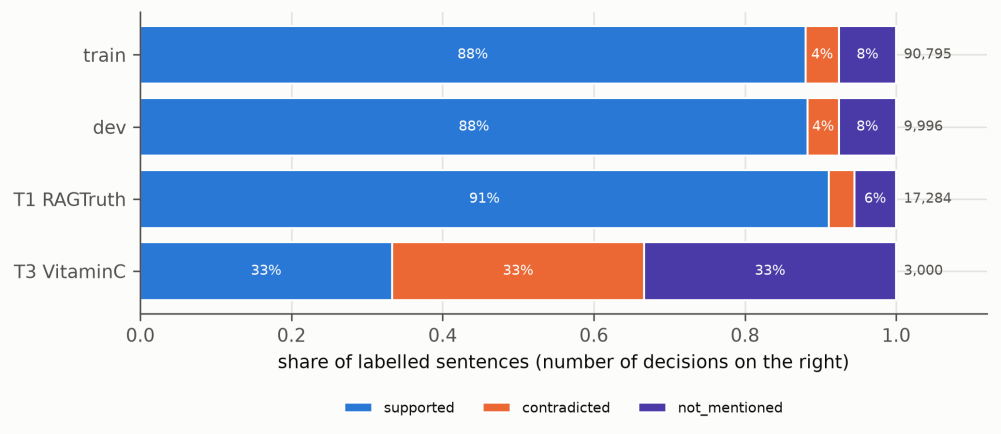


Figure 5.1: Class balance of the labelled sentences in each split. RAGTruth (train, development and T1) is about 88–91 % supported; T3 is sampled to 1,000 pairs per class.

5.5 Reliability of the test sets

The test sets were checked without new annotation, using evidence that is independent of the construction described above (Table 5.3). All test labels come from the human annotators of the published datasets. The two choices that could introduce errors, segmentation and the conversion from spans to sentence labels, were compared with LLM-AggreFact, whose authors converted the same RAGTruth test answers into sentence-level claims independently. The segmenter recovers 96.3 % of their claims exactly, and only 8.5 % of the scored sentences are absent from their

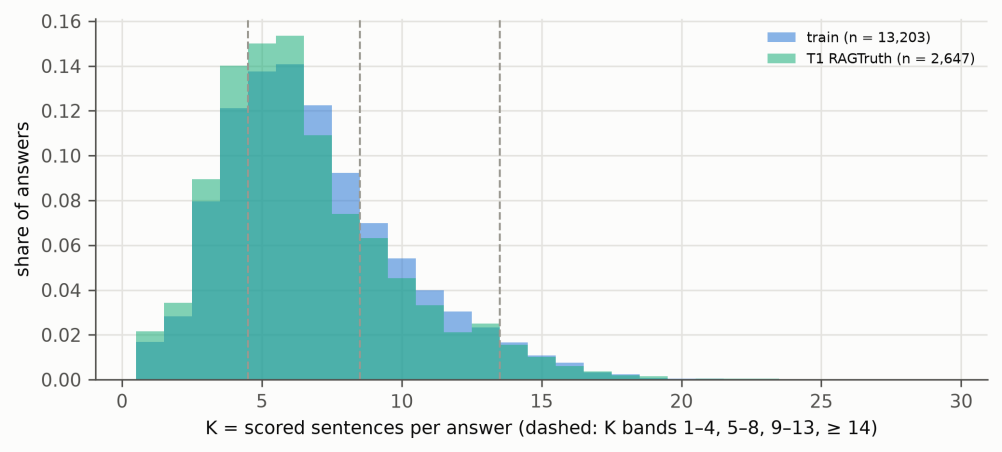


Figure 5.2: Distribution of K, the number of scored sentences per answer, in training and in T1. Dashed lines mark the bands used to report results as a function of K.

set, mostly sentences that their undocumented filter of "non check-worthy" claims removed. On the 15,492 sentences present in both, the binary labels agree in 98.75 % of cases, and all 193 disagreements go in the same direction: the span rule marks a sentence as unsupported where LLM-AggreFact considers it supported. Finally, an encoder baseline evaluated on the unchanged T2 reproduces its published score (74.0 against 73.5), which validates the evaluation code.

Table 5.3: Reliability checks run before the test sets were frozen.

Check	Criterion	Result
Label provenance	every test label comes from human annotators	pass
Segmentation	$\geq 90\,\%$ of LLM-AggreFact claims recovered, $\leq 10\,\%$ of sentences absent from it	96.3 % / 8.5 %
Label derivation	$\geq 97\,\%$ agreement with LLM-AggreFact's binary labels	98.75 %
Reference baseline	MiniCheck-RoBERTa on T2 within 1.5 points of its published score	74.0 vs 73.5
Leakage	no shared source, document or (document, sentence) pair between training and test	pass

These checks validate the binary side of T1. The distinction between contradicted and not mentioned rests on the RAGTruth annotators, whose agreement on the hallucination type is not published. Moreover, a recent re-annotation of the RAGTruth test split found 1.68 times more hallucinated answers than the original

labels [114], which suggests that T1 under-counts the two minority classes. For these reasons the three-way results on T1 are always read together with those on T3. After these checks, and before any model was trained, the three test files were frozen with cryptographic hashes, and the dataset builder refuses to overwrite them with different content; rebuilding the dataset from the raw sources reproduces the frozen files exactly.

5.6 Exploratory analysis

An exploratory analysis of the sources and of the built dataset revealed four properties that shape how the results must be read.

Errors depend on the task and on the model. Data-to-text answers are hallucinated two to four times as often as answers in the other tasks, and answers written by Llama-2 and Mistral contain most of the hallucinations, while GPT-3.5 and GPT-4 answers are almost always faithful (Figure 5.3). Results are therefore reported per task, since a single average would mostly reflect data-to-text.

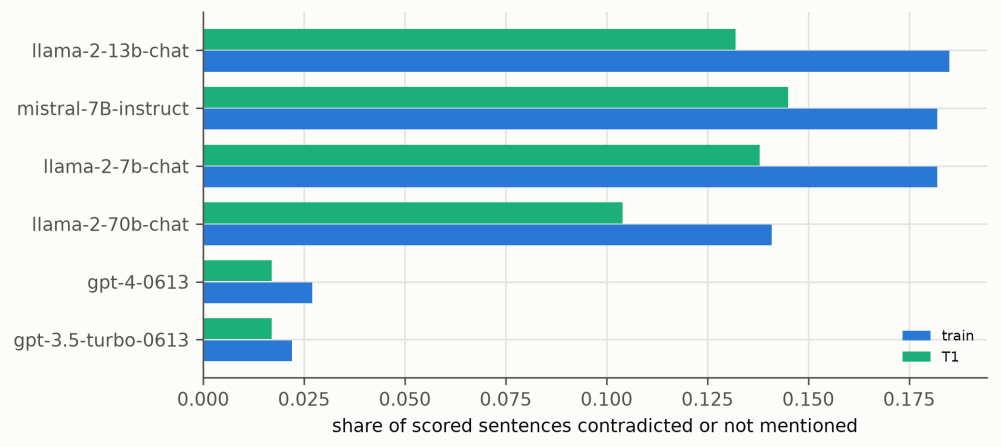


Figure 5.3: Share of contradicted or not-mentioned sentences by the model that wrote the answer, in training and in T1.

The test answers are cleaner than the training answers. In every task, and for every generator, T1 has a lower share of contradicted and not-mentioned sentences than the training set; in question answering, for instance, 0.6 % of T1 sentences are contradicted against 1.6 % in training. Metrics that depend on the base rate, such as precision or F1 on the minority classes, are therefore not

comparable between development and test. The primary metrics are balanced accuracy, macro recall and AUROC, which do not depend on it.

Contradictions are mostly evident. About 85 % of the contradicted training sentences come from spans annotated as *evident* conflicts, such as wrong numbers or names, and only 147 from *subtle* conflicts. The contradicted class is therefore mostly blatant contradictions, and subtle contradictions cannot be evaluated separately.

Surface cues exist and define the baselines. Two simple cues predict the labels without verifying the sentence against the document. The first is word overlap. The share of a sentence's content words that also appear in the document is higher for supported than for contradicted and not-mentioned sentences, in all three test sets (Figure 5.4). On its own it separates supported from unsupported sentences with an AUROC of 0.74 on T1 and 0.70 on T3. The second cue is the wording of the sentence. A classifier that sees only the sentence reaches a three-way balanced accuracy of 0.66 on data-to-text, because some details, such as the number of reviews or the availability of Wi-Fi or valet parking, are often invented in Yelp descriptions. In question answering and summarisation the same classifier stays at chance (0.36 and 0.35), and on T3 as well (0.33), where the contrastive design removes the cue. A verifier is useful only to the extent that it beats these cues, so both are reported as baselines, per task.

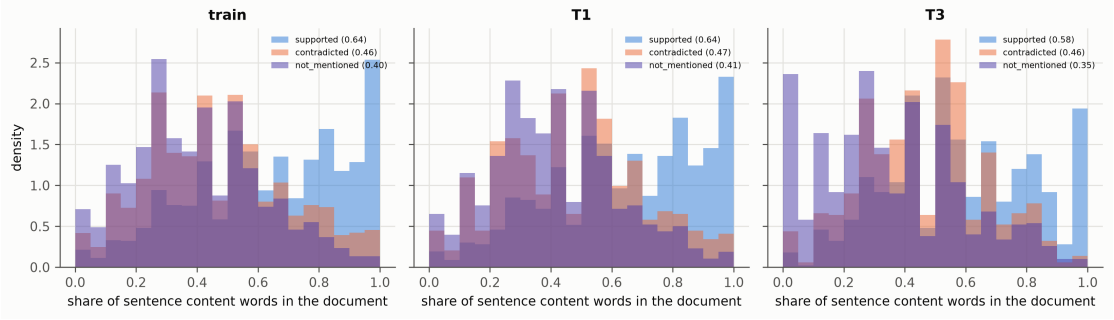


Figure 5.4: Word overlap between a sentence and its document, by class, in training, T1 and T3 (mean in brackets).

Finally, unsupported sentences are not concentrated at the end of an answer. The first sentence is rarely unsupported, the share peaks at positions three to five and stays between 8 and 11 % in later positions (Figure 5.5). Long answers still contain errors near their end, so accuracy at high K matters.

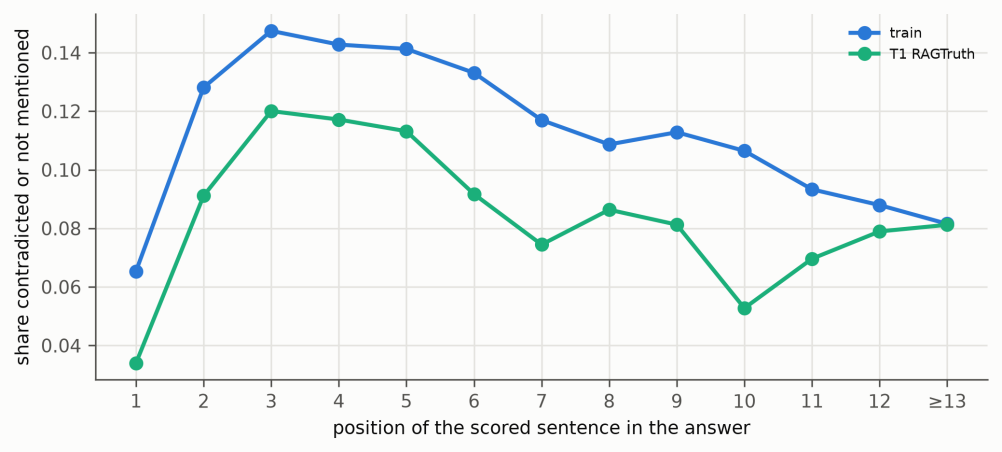


Figure 5.5: Share of contradicted or not-mentioned sentences by position in the answer.

5.7 Leakage control

Splits are made by source, never by sentence, and an automatic check runs on every build. It fails if a source, a document or a (document, sentence) pair of the training or development set appears in any test set, and if the development and training sets share a source. The check passes. It also reports sentence strings that occur in both training and test with different documents: these are 788 template sentences, such as “The business has a rating of 4.0 stars.”, which are expected in data-to-text and do not reveal any test label. Whether the public datasets used here were part of the pre-training data of Qwen2.5 cannot be verified, and this remains a limitation shared by most work on public benchmarks.

Chapter 6

Experimental Setup

This chapter describes the models that were trained, the systems they are compared with, the metrics, the statistical protocol and the way time and energy are measured. All experiments run on one machine, a MacBook Pro with an Apple M4 chip (10-core CPU, 10-core GPU, 120 GB/s of memory bandwidth) and 16 GB of unified memory [43], so that every result reflects what a verifier can do on the device where a local assistant runs. Every baseline, including the encoders and the two general-purpose System One models, also runs on this laptop; no result depends on a remote service.

6.1 Models

Both GroundOne models start from the instruction-tuned Qwen2.5 checkpoints [23] converted to MLX with 8-bit weights, and both are trained with the recipe of Section 4.3. They differ only in size, in the number of training answers and in how micro-batches are formed, which memory dictates.

The 0.5B model is trained on all 13,203 training answers. One epoch of 802 optimiser steps took 5.5 hours of wall-clock time, with an evaluation on the full development set every 150 steps, and training used at most 2.6 GB of memory. The 3B model is about six times slower per token, so it is trained for one night on a random subset of 6,500 answers (about 45,000 decisions): 403 steps in 12.25 hours, with a peak of 5.9 GB. Its development evaluation, every 80 steps, uses a fixed subset of 200 answers, because a full pass would take about an hour. In both cases the checkpoint with the best development score is kept and then calibrated as described in Section 4.4; for the 3B this is step 320.

To measure how much the results depend on the random seed, which sets the initialisation of the adapters and the order of the batches, the 0.5B recipe is repeated with a second seed. Fine-tuning outcomes are known to vary with both factors [96].

One further run with a third seed collapsed early to a constant prediction and was stopped at about 60 % of the epoch; being incomplete, it is excluded from the tables, and it is discussed in Section 7.2. The completed second run is evaluated on all three test sets exactly like the first.

6.2 Baselines

The baselines isolate the sources of any advantage. Each one answers a specific question.

Cues that never read the document as a whole. Two simple predictors give the floor that any verifier must clear. The first scores a sentence by the share of its content words that also occur in the document. The second is a bag-of-words classifier trained on the training sentences alone, without the document. Both are described in Section 5.6.

Untrained readout (B1). The same Qwen2.5 models, without fine-tuning, are asked the three-way question for each sentence and their answer-letter logits are read. Because an untrained model has no reason to treat the `Verdict:` marker as a question, this baseline uses the format with one explicit question per sentence (Section 4.2). It measures what the System One mechanism gives before any specialisation.

Generative judges (B0a, B0b). The untrained models are also used in the way that is common in practice: they receive the document, the numbered answer and the list of sentences to check, and they write a verdict for each sentence, with greedy decoding. The *terse* judge writes one line "`<number>: <letter>`" per sentence; the *verbose* judge writes one short sentence of reasoning followed by the verdict. A verdict that cannot be parsed is a format failure; it is counted as an error in accuracy and reported separately.

Equal-output judges (B0d). The terse and verbose judges return less than GroundOne: a letter per sentence and no probability, so they offer no calibration, no adjustable threshold and no choice of policy. Two further judges are therefore asked for exactly GroundOne's content, a label and three probabilities per sentence. The first writes them as a JSON object, with the probabilities expressed in words, which is what a developer would obtain from a prompt alone. The second is the terse judge whose three probabilities are read from the logits at the letter it generates; it obtains the same content at the cost of the terse judge and is the strongest generative opponent. Both judges are calibrated with the same temperature and

bias as GroundOne, fitted on 200 development answers, so that their probabilities receive the same treatment. The JSON prompt was chosen on twelve development answers for format validity only: a numeric example in the prompt was copied verbatim by the models and was replaced by placeholders. Parsing is lenient, in the judges' favour: complete entries of a truncated object are kept, and a missing label is taken from the probabilities. Decoding is not constrained by a grammar [87], so that the protocol stays that of the other judges; this choice affects mainly the format failures of the smallest model.

Readout versus generation (B2). The fine-tuned GroundOne models are run with the same prompt and the same single pass, but the verdict at each marker is the token with the highest probability over the whole vocabulary, which is what greedy generation of one token would produce. The three letters, with or without a leading space, map to the three classes, and any other token is a format failure. This control keeps weights, prompt and cost identical and changes only whether the decision is restricted to the admissible options.

A general-purpose System One: Rizzo-Flow (B3). This open project [16] implements the System One idea with a 4B model fine-tuned on generic typed decisions. It runs in its own server, based on llama.cpp [2], and is queried over HTTP with the document and the numbered answer as shared state and one three-way choice question per sentence. A 4-bit quantised model is used because the 8-bit one does not fit in memory beside the operating system.

A general-purpose System One of the same size: d1-3B (B3b). Liquid AI's d1-3B [18] is an open decision model of 3.1B parameters, post-trained for typed decisions on a broad mixture of tasks, which reads restricted answer letters with the state shared across questions: the same readout family as GroundOne, at the size of its larger model, but general-purpose. It isolates the value of specialisation against a System One that was already trained to decide, rather than against an untrained one. It is run zero-shot with its own inference code, in PyTorch on the GPU of the laptop and without network access, with the same state and questions as Rizzo-Flow and at most six questions per call on the same state; on LLM-AggreFact its documents are split exactly as GroundOne's and the maximum over chunks is kept. A practical detail matters for reproducibility: PyTorch keeps compiled kernels for every new input length on the Apple GPU and does not release that memory, so the evaluation runs in worker processes that are restarted when the memory used by the GPU driver exceeds 11 GB, which leaves the outputs unchanged.

Specialised encoders (B4). MiniCheck-RoBERTa [12] and HHEM-2.1-open [13] are the strongest small verifiers available. They score each (document, sentence) pair independently and return one probability of support; long documents are split into chunks of about 400 words and the maximum over chunks is kept, as in MiniCheck. They have no three-way output, so they are compared on the binary decision only. They run with PyTorch on the GPU of the laptop.

Published results. For LLM-AggreFact, the scores of larger systems from the public leaderboard, such as Qwen2.5-7B used zero-shot, are reported as given and not re-run.

The slower systems are evaluated on stratified subsamples of the test sets: RizzoFlow on 300 answers of T1, 600 pairs of T3 and 1,100 items of T2, of which it scored 1,093 (seven exceeded its context); d1-3B on a random subsample of 4,000 items of T2; the untrained 3B on 600 answers of T1; and the generative judges on the 200 answers of the latency protocol below. GroundOne, the untrained 0.5B, the encoders and, on T1 and T3, d1-3B are evaluated on the complete test sets, and the comparisons with a subsampled system are always made on the items that both systems scored.

6.3 Metrics

Because the share of hallucinated sentences differs between the development set and T1 (Section 5.6), the primary metrics do not depend on the base rate. For the three-way decision the main metric is the balanced accuracy, that is the mean recall over the three classes, together with the recall of each class and the share of each predicted class, which reveals a model that collapses onto one answer. For the binary decision the metrics are the balanced accuracy at a threshold of 0.5, as in the LLM-AggreFact leaderboard, and the AUROC of $P(\text{supported})$, which does not depend on any threshold. Calibration is measured with the ECE defined in Section 4.4, the Brier score and the negative log-likelihood, and selective prediction with risk–coverage curves and the area under them (AURC) [95]. All metrics on T1 are also reported per task and per band of K.

6.4 Statistical protocol

Every difference between two systems is accompanied by a 95 % confidence interval from a paired bootstrap with 2,000 resamples [115, 116, 117]. The resampling unit is not the sentence but the group that the leakage check uses: the source document in RAGTruth, the document in LLM-AggreFact and the Wikipedia page in VitaminC. Sentences of the same answer, and answers about the same document, are strongly

correlated, and resampling them independently would produce intervals that are too narrow. On LLM-AggreFact the groups are resampled within each of the eleven datasets, because the macro average gives each dataset the same weight. Both systems of a comparison are evaluated on the same resamples, and the reported p-value is the two-sided share of resamples in which the difference changes sign.

Two limits of this protocol are stated in advance. First, about fifty comparisons are tested and no correction for multiple comparisons is applied; the conclusions therefore rest on intervals that lie far from zero, and a difference with a p-value near 0.05 is not claimed. Second, resampling the data does not capture the variability due to training: the two runs of the 0.5B provide that information separately (Section 7.2).

6.5 Latency, energy and hardware

Latency is measured on 200 answers of T1, 50 from each band of K, so that long answers are as well represented as short ones. GroundOne, the question-per-sentence readout and all generative judges run in process on MLX, one answer at a time, as a verifier would in a local assistant. The first three answers serve as warm-up and are excluded. For the System One variants each answer is timed three times and the median is kept; the generative judges are timed once, since greedy decoding is deterministic and each run is much longer. The reported time is the wall-clock time of the model call for one answer, from which the time per decision follows.

Times were also recorded for the encoders, for Rizzo-Flow and for d1-3B, but under conditions that do not allow a fair comparison. The encoders ran with PyTorch and scored each sentence separately, re-reading the document every time, which penalises them; Rizzo-Flow ran in a different inference engine, with 4-bit weights and behind an HTTP interface; d1-3B ran in PyTorch with its own code and with the worker restarts described above. No latency claim is therefore made against them.

Energy is measured with the `powermetrics` tool of macOS, which reports the power of the CPU, the GPU and the Neural Engine every 200 milliseconds. For each system, 100 answers of the latency subset (25 per band of K) are verified once, after 60 seconds of idle measurement with the model already loaded; the energy of a verification is the mean power during the run, minus the idle power, multiplied by the run time and divided by the number of answers. The measurement covers the computing units of the chip but not the memory, the display or the storage, and it was taken overnight with the machine otherwise idle.

Only one model runs at a time, and a memory guard runs every long job and records the free memory and the swap throughout. Results are written once

and never overwritten, and each run stores its configuration, so every number in Chapter 7 can be traced to the run that produced it.

Chapter 7

Results

This chapter reports how GroundOne learns and how stable its training is, what it achieves on the three frozen test sets, how it compares with the baselines, and what the System One readout contributes by itself. It then examines the quality of the probabilities, the time and energy needed to verify an answer, and the errors that remain. Intervals are 95 % cluster-bootstrap intervals computed as described in Section 6.4.

7.1 Training

Both models learned the task. Before training, a model reading the marker format has no notion of what the marker asks and its verdicts are at chance: the 0.5B answers "contradicted" for 88 % of the development sentences and reaches a three-way balanced accuracy of 0.32. In the completed runs the training loss fell below the value of a model that learned only the class frequencies ($\ln 3 \approx 1.10$) within the first hundred steps, and the development score then rose steadily (Figure 7.1). The 0.5B, trained on all 13,203 answers, reached its best score at step 600 of 802, with a balanced accuracy of 0.633 averaged over the three tasks, after which it stayed flat. The 3B, trained for 12.25 hours on 6,500 answers, reached 0.705 at step 320 of 403 on its development subset. Measured on the full development set, the three-way balanced accuracy is 0.731 for the 0.5B and 0.784 for the 3B, and all three classes were predicted throughout both runs.

7.2 Variability across seeds

The 0.5B recipe was run again with a second seed, which changes the initialisation of the adapters and the order of the batches. When training converges, it converges to the same place (Figure 7.2): the second run reaches a three-way balanced

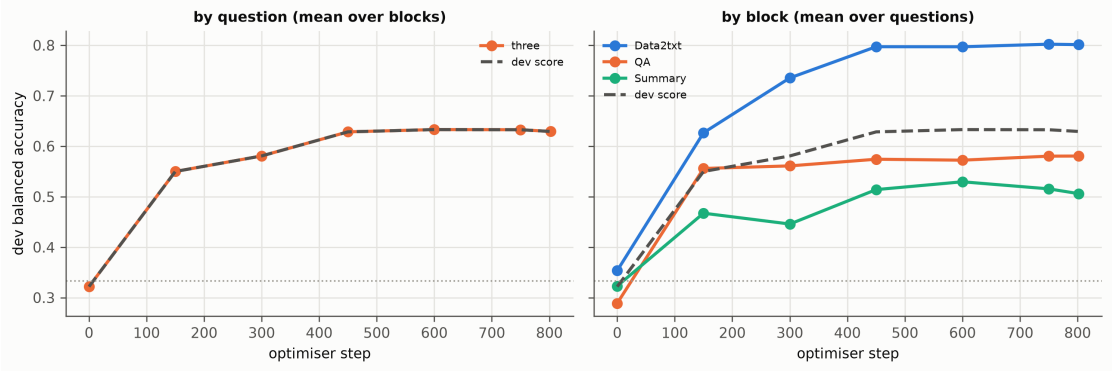


Figure 7.1: Three-way balanced accuracy on the development set during training of the 0.5B model, overall (left) and per task (right). The dotted line marks chance. The 3B shows the same shape on its 200-answer development subset.

accuracy of 0.767 on T1 against 0.769, an AUROC of 0.907 against 0.905, 71.5 against 70.1 on LLM-AggreFact and 0.434 against 0.442 on VitaminC. On T1 the paired difference is +0.001 (−0.008 to +0.011) and is not significant. The variation lies in finer details. The errors are distributed differently between the classes, with the recall of supported and not-mentioned sentences moving by about 0.04 in opposite directions, and the smaller slices move more: the band with fourteen or more decisions by 0.023, the AUROC on VitaminC by 0.025 and single datasets of LLM-AggreFact by up to ten points, as on WiCE (67.9 against 78.0).

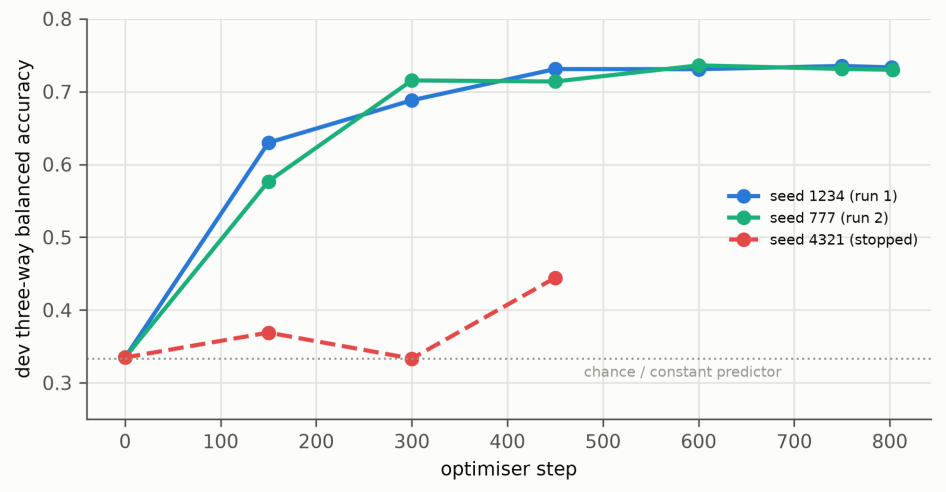


Figure 7.2: Development three-way balanced accuracy of the 0.5B during training with the same recipe and three seeds. Two runs converge to the same level; the third collapsed to a constant predictor and was stopped.

Two consequences follow. First, the bootstrap, which resamples the test data, does not capture this source of variability: on LLM-AggreFact and VitaminC the two runs of the same recipe differ by more than their intervals allow (-0.013 on the macro of T2, -0.025 on the AUROC of T3). Differences between systems that are smaller than the spread between seeds, about 1.5 points on the macro of LLM-AggreFact and 0.025 in AUROC on VitaminC, are therefore not interpreted, and neither are single datasets of LLM-AggreFact unless the gap exceeds about five points.

Second, training does not always converge. A run with a third seed stayed near chance: its development score was 0.369 at step 150 and 0.333 at step 300, where it predicted "supported" for every sentence, and the training loss remained close to the value of a model without information. The run was stopped at step 470 of 801, when its score had climbed to 0.444, so whether it would have recovered by the end of the epoch is unknown; being incomplete, it is excluded from the tables. The recipe has two features that are known to make fine-tuning fragile: gradients are not clipped, and the optimiser runs without bias correction, the default of the framework, a setting that has been identified as a main cause of failed fine-tuning runs [98, 97]. One collapse in three runs does not measure a failure rate, but it shows that the recipe is not fully robust to initialisation.

7.3 Accuracy on the RAG test set (T1)

On the 17,284 sentences of T1 the 0.5B model reaches a three-way balanced accuracy of 0.769 and the 3B model 0.832 (Table 7.1); the difference, $+0.063$ ($+0.052$ to $+0.076$), is the clearest effect of model size. On the binary decision the 0.5B reaches a balanced accuracy of 0.815 and an AUROC of 0.905, and the 3B 0.874 and 0.940. Both are far above the cues that never verify a sentence against its document: word overlap reaches an AUROC of 0.737, and the sentence-only classifier a three-way balanced accuracy of 0.570.

Three observations stand out. First, accuracy does not degrade with the length of the answer: answers with fourteen or more decisions are among the best handled, with 0.723 for the 0.5B and 0.839 for the 3B, and the hardest band is the shortest, which mostly contains question-answering and summarisation answers. This matters because long answers are where a System One saves the most time; on these answers the gain of the 3B over the 0.5B is $+0.116$ ($+0.050$ to $+0.180$). Second, contradictions are recognised mainly in data-to-text, where RAGTruth concentrates them. In question answering and summarisation the 0.5B separates supported from not-mentioned sentences well but finds almost no contradictions, with a recall of 0.03 and 0.13. The 3B raises both to 0.30, so model size compensates in part for the scarcity of such examples in the training data. Third, question

Table 7.1: GroundOne on T1, by task and by number of decisions per answer (K): three-way balanced accuracy, recall of contradicted sentences, and AUROC of P(supported).

		GroundOne 0.5B			GroundOne 3B		
Subset	Sentences	Bal. acc.	Contr.	AUROC	Bal. acc.	Contr.	AUROC
All	17,284	0.769	0.68	0.905	0.832	0.76	0.940
Question answering	4,876	0.589	0.03	0.916	0.699	0.30	0.948
Summarisation	4,538	0.544	0.13	0.775	0.629	0.30	0.873
Data-to-text	7,870	0.812	0.84	0.929	0.873	0.90	0.955
$K = 1$–4	2,434	0.606	0.18	0.857	0.677	0.25	0.921
$K = 5$–8	8,012	0.758	0.67	0.899	0.829	0.78	0.936
$K = 9$–13	5,201	0.824	0.82	0.926	0.863	0.86	0.945
$K \geq 14$	1,637	0.723	0.50	0.908	0.839	0.68	0.957

answering and summarisation are the clearest evidence that the models read the document: there the sentence-only classifier stays at chance (0.36 and 0.35), while the two models reach 0.59–0.70 and 0.54–0.63.

The same predictions can be read at the level of whole answers, which is how the community Decision Index scores general-purpose decision models on RAGTruth: an answer counts as hallucinated if it contains any unsupported content, and the metric is the F1 score of that class, which a system answering "hallucinated" every time already reaches at 0.52 [22]. Taking the minimum calibrated P(supported) over the sentences of an answer, with a threshold fitted on the development set, GroundOne reaches 0.733 with the 0.5B and 0.784 with the 3B on the 2,647 answers of T1. On the leaderboard no general decision model below one billion parameters exceeds 0.55, Jev reaches 0.765 and the best model, of 27B parameters, 0.803. The comparison is only indicative, because GroundOne was trained on RAGTruth while the leaderboard models are evaluated zero-shot, with a different prompt and on 53 more answers, but it places a 0.5B specialist far above general decision models of its size.

7.4 Comparison with the baselines

Figure 7.3 compares all systems on T1. The specialised encoders reach an AUROC of 0.814 (MiniCheck-RoBERTa) and 0.821 (HHEM), above word overlap but below both GroundOne models: the 0.5B exceeds them by +0.091 (+0.074 to +0.107) and +0.083 (+0.066 to +0.099), the 3B by +0.126 and +0.119. The comparison by task qualifies this result. GroundOne's advantage comes mostly from data-to-text, where it is trained in domain and where the encoders are weakest (AUROC 0.71–0.75),

while on summarisation MiniCheck-RoBERTa is better than the 0.5B model (0.872 against 0.775) and level with the 3B (0.873). The untrained models show how much of the accuracy training provides: the 0.5B is at chance because it never answers "supported", and the 3B, asked the same question, reaches 0.437 against 0.818 after training, measured on the same answers. How much the restricted readout contributes on top of training is measured separately in Section 7.6.

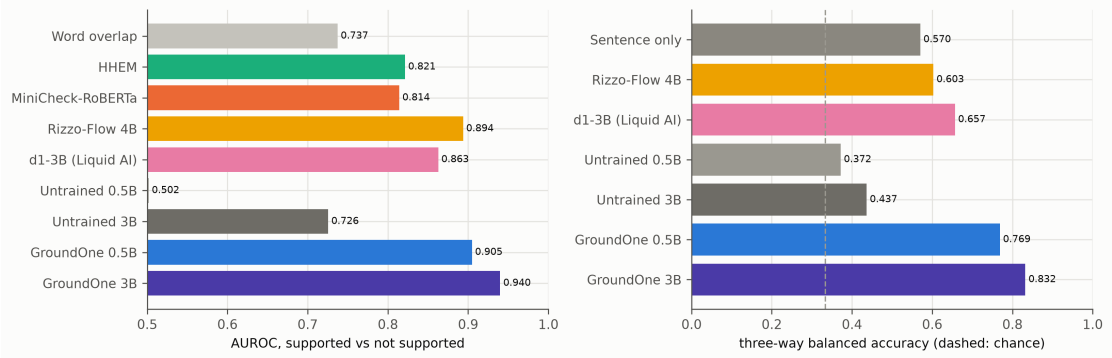


Figure 7.3: T1: AUROC of P(supported) for every system (left), and three-way balanced accuracy for the systems with a three-way output (right). Rizzo-Flow is evaluated on 300 answers and the untrained 3B on 600; all other systems, including d1-3B, on the full test set.

Rizzo-Flow, the larger general-purpose System One, gives the most instructive comparison. On the same 300 answers of T1, Rizzo-Flow, a 4B model fine-tuned for generic decisions, reaches a three-way balanced accuracy of 0.603, against 0.798 for the 0.5B GroundOne and 0.840 for the 3B; the differences, +0.195 (+0.136 to +0.249) and +0.237 (+0.184 to +0.285), are large. Its AUROC (0.894) is close to that of the 0.5B on these answers (0.916; the difference is not significant) because it ranks sentences well but predicts "supported" for 90 % of them; the 3B ranks them better (+0.056, +0.034 to +0.079). A specialist eight times smaller therefore outperforms the generalist on the decision it was built for, while matching it in the ability to rank sentences.

d1-3B, a general-purpose System One of the same size as the 3B, gives the sharpest test of specialisation, because it was already trained to make typed decisions and differs from GroundOne mainly in what it was trained on. On the full T1 it reaches a three-way balanced accuracy of 0.657 and an AUROC of 0.863. Both GroundOne models are clearly better: the 0.5B, six times smaller, by +0.112 (+0.092 to +0.133) in three-way accuracy and +0.042 in AUROC, the 3B by +0.175 (+0.157 to +0.195) and +0.077. The gap is widest on long answers, where d1-3B falls to 0.527 with fourteen or more decisions and the 3B GroundOne stays at 0.839 (+0.312, +0.230 to +0.405), and d1-3B recognises only half of the

not-mentioned sentences (recall 0.49). Within the domain of retrieval-augmented answers, specialisation is therefore worth far more than a general training for decisions.

On LLM-AggreFact (T2) the 0.5B model reaches a macro-averaged balanced accuracy of 70.1 over the eleven datasets, against 64.2 for the same model used zero-shot and 72.8 for the zero-shot 7B model on the public leaderboard. It remains below MiniCheck-RoBERTa (74.0) by 3.9 points, a significant difference. On the four datasets closest to retrieval-augmented generation the two are on par (74.4 against 74.6), and on the RAGTruth column GroundOne leads, 82.1 against 76.5, a comparison that favours it because it was trained on RAGTruth training answers. Outside retrieval-augmented generation the 0.5B is weaker than the encoder by seven to nine points on AggreFact-CNN, AggreFact-XSum, FactCheck-GPT and REVEAL, the expected cost of training on RAG data only.

The 3B model, evaluated on the full test set, closes most of this gap. Its macro-averaged balanced accuracy is 74.8, on par with MiniCheck-RoBERTa (74.0; paired difference +0.8 points, interval −0.5 to +2.1) and above the published zero-shot score of the 7B model (72.8, for which no interval can be computed). On the RAG datasets it reaches 76.9, 2.4 points above the encoder (interval +1.4 to +3.3). Its gain over the 0.5B, +4.7 points (+3.4 to +5.9), is broad, with the largest improvements on REVEAL, WiCE, FactCheck-GPT and XSum, so model size improves transfer beyond the training domain. The 3B remains clearly below the encoder only on news summarisation (AggreFact-CNN, 59.3 against 67.1). Rizzo-Flow, on the 1,093 items of its subsample, reaches 61.5; there the 3B GroundOne is 12.0 points better (+7.4 to +16.5). An independent evaluation of the closed System One Jev on the same test set and with the same metric reports 78.6, and 76.9 for a 27B open model read through its answer letters, with prompts different from those used here, so that no interval can be computed [20]. On a broad grounding benchmark, then, a 3B specialist trained only on RAG answers matches the best small encoder but remains below the closed state of the art and much larger open models. The general-purpose d1-3B confirms the picture at the same size: on a random subsample of 4,000 items it reaches 75.7, against 74.3 for the 3B GroundOne on the same items, a difference that is not significant (−1.3, −4.0 to +1.0), while it beats the 0.5B by 6.5 points. It is weaker on the RAGTruth column (78.9 against 87.3) and clearly stronger on WiCE and on dialogue summarisation.

7.5 Transfer to a different domain (T3)

On VitaminC the picture reverses. On the full test set the 0.5B reaches a three-way balanced accuracy of 0.442 and the 3B 0.527, with AUROCs of 0.722 and 0.817, and MiniCheck-RoBERTa reaches an AUROC of 0.968. On the 600 pairs evaluated

with Rizzo-Flow, the general-purpose model reaches 0.712 and an AUROC of 0.909, against 0.442 and 0.744 for the 0.5B and 0.537 and 0.845 for the 3B. Two facts put these numbers in context. MiniCheck-RoBERTa starts from the AlignScore model, whose training data include VitaminC [11], so T3 belongs to its training distribution, whereas for GroundOne it measures transfer from RAGTruth. Moreover, the gain of the 3B over the 0.5B on T3 comes from model size rather than from the RAG training: the untrained 3B already reaches 0.531 and an AUROC of 0.821. d1-3B, the general-purpose System One of the same size, makes the contrast explicit: it reaches 0.724 and an AUROC of 0.913 on the full T3, ahead of the 3B GroundOne by 0.197 and 0.096, the reverse of what happens on T1. Specialisation on RAG answers therefore improves the RAG task substantially but does not, by itself, produce a general fact checker. Section 7.11 shows which cases are missed.

7.6 Readout versus generation

The control B2 isolates the readout. With the same fine-tuned weights, the same prompt and the same single pass, the verdict at each marker is either read from the three answer letters, as GroundOne does, or taken as the token that greedy generation would emit (Figure 7.4). For the 0.5B the generated token gives a three-way balanced accuracy of 0.521 against 0.769 for the readout, a difference of +0.248 (+0.228 to +0.269): in a quarter of the markers (24.7 %) the model would not emit a letter at all, but most often a bare space, the beginning of a longer answer. For the 3B the generated token is almost always a letter (1.2 % failures) and agrees with the readout in 92 % of the markers, yet the readout is still better, 0.832 against 0.761 (+0.071, +0.059 to +0.083). On answers with fourteen or more decisions the gap is largest for the small model, +0.346.

The latency of the two options is the same, since both use one pass; what differs is the decision rule. Restricting the choice to the admissible options removes format failures by construction and, since the model is trained only to order the three letters, also gives the more accurate decision (Section 4.2). The smaller the model, the more this restriction matters. The often repeated expectation that reading answer logits and generating a single answer token are equivalent therefore holds for the cost but not for the accuracy of a fine-tuned verifier.

7.7 Calibration and selective prediction

The raw probabilities underestimate support. In the reliability diagram (Figure 7.5) the raw curve lies above the diagonal, because the class weights train the model as if the classes were balanced; the binary ECE on T1 is 0.30 for the 0.5B and 0.18 for the 3B. The temperature and bias fitted on the development set bring the ECE

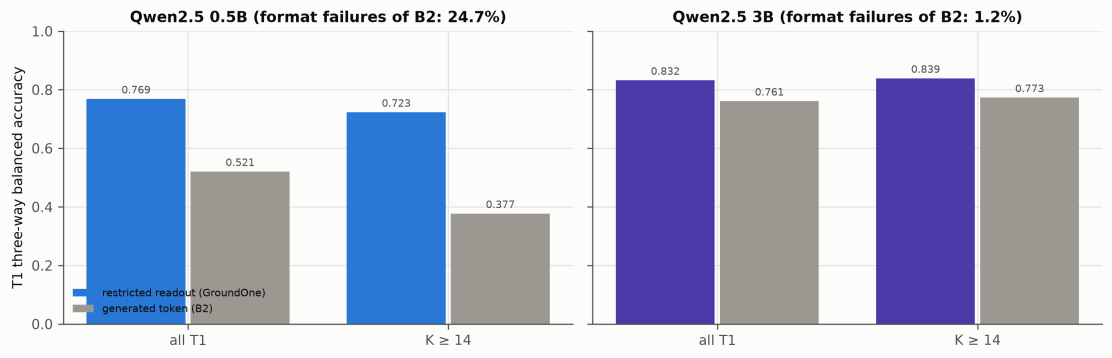


Figure 7.4: Control B2 on the full T1: with the same weights and the same forward pass, the verdict is read from the three answer letters (GroundOne) or taken as the token that greedy generation would emit (format failures counted as errors).

on T1 to 0.019 for both models, and to 0.020 for the second run of the 0.5B, and reduce the negative log-likelihood from 0.55 to 0.21 (0.5B) and from 0.36 to 0.17 (3B), while the AUROC is practically unchanged. The map fitted on development data therefore transfers to the test set of the same domain.

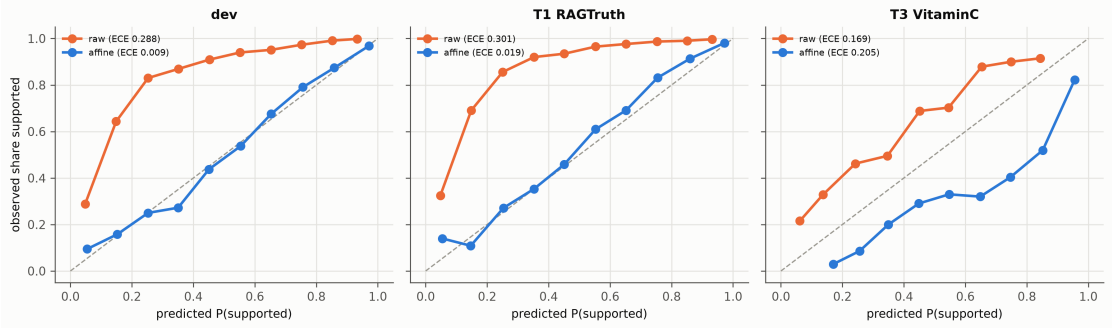


Figure 7.5: Reliability of P(supported) of the 0.5B before (raw) and after calibration on the development set, for the development set, T1 and T3. Points are bins with at least 20 sentences.

Calibration has a consequence that must be kept in mind. The bias restores the true frequency of each class, so the most probable calibrated class becomes "supported" more often: for the 0.5B on T1 the accuracy rises from 0.82 to 0.93, but the three-way balanced accuracy falls from 0.77 to 0.63. Taking the most probable calibrated class is therefore not a good decision rule when the three kinds of error matter equally. The served rule of Section 4.4 resolves this: with the threshold fitted on development data, $\tau = 0.935$ for the 0.5B, 0.895 for its second run and

0.925 for the 3B, it reaches 0.758, 0.769 and 0.830 on T1, within 0.01 of the raw decision evaluated throughout this chapter, while keeping labels and calibrated probabilities consistent. The thresholds are high because 91 % of the RAGTruth sentences are supported: a sentence is served as supported only when the model is very confident.

Used as confidences, calibrated probabilities make selective prediction effective: deferring the least confident sentences lowers the error rate of the binary decision quickly, with an area under the risk–coverage curve of 0.015 (0.5B) and 0.010 (3B) on T1 (Figure 7.6).

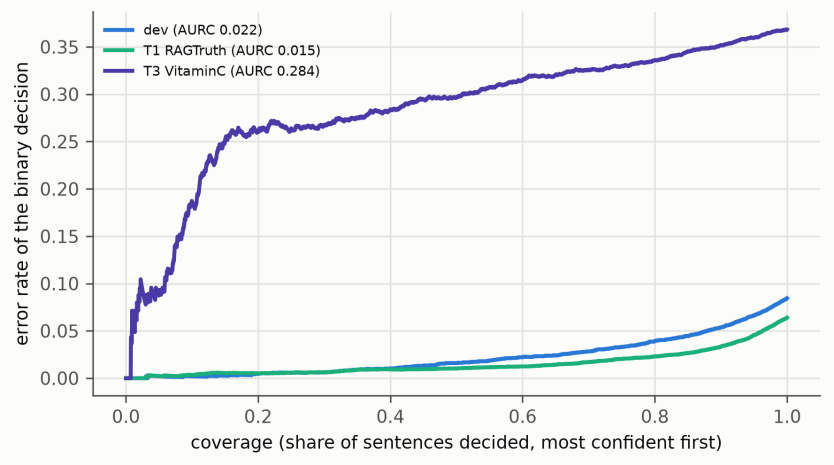


Figure 7.6: Risk–coverage curves of the binary decision of the 0.5B with calibrated probabilities: error rate as the least confident sentences are deferred.

Calibration does not transfer to VitaminC, where the binary ECE rises from 0.17 to 0.21 for the 0.5B and from 0.14 to 0.19 for the 3B, and where the served rule loses 0.02–0.04 with respect to the raw decision. The bias encodes the base rate of RAGTruth, while VitaminC is balanced by construction. A calibration is valid for the distribution it was fitted on, so a deployment needs a small labelled sample of its own data.

7.8 Probabilities at equal output

The equal-output judges answer whether a generative judge of the same size, asked for the same content as GroundOne, can match the quality of its probabilities. It cannot (Figure 7.7). On the 200 answers of the latency subset, with every system calibrated on development data in the same way, the AUROC of P(supported) is 0.94 and 0.97 for GroundOne, 0.48 and 0.68 for the judge whose probabilities are read from the logits, and 0.47 and 0.58 for the judge that writes them, for the

0.5B and the 3B respectively. The area under the risk–coverage curve is 0.009 and 0.005 for GroundOne against 0.048 to 0.063 for the judges, five to twelve times larger. The accuracy of the decisions follows the same pattern: 0.835 and 0.900 for GroundOne against at most 0.41 for any judge.

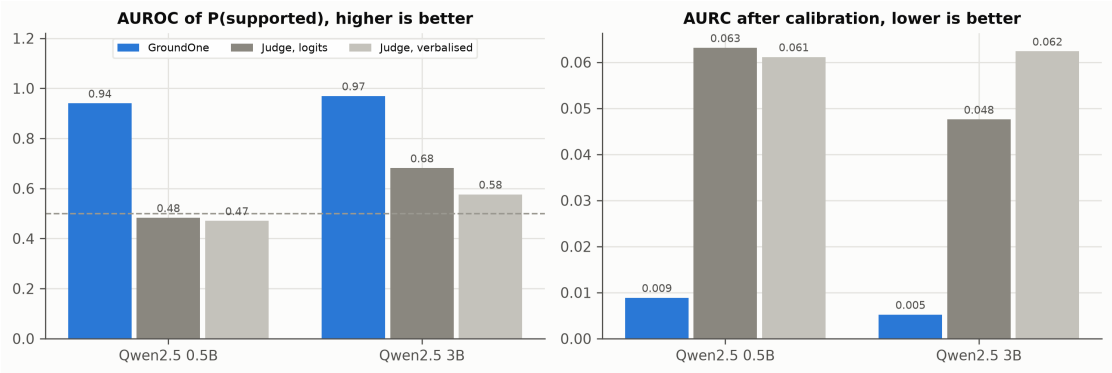


Figure 7.7: Equal-output comparison on the 200 latency answers: generative judges of the same size asked for GroundOne's output, with probabilities read from the logits of the generated letter or written in JSON; all systems calibrated on development data with the same map. Sentences with a valid verdict only.

Two further observations matter for practice. Probabilities written in words carry almost no information: the 3B writes a vector with all the mass on one class for 93 % of the sentences, using only sixteen distinct vectors, and the 0.5B writes uninformative ties such as equal probabilities for all classes; counting the ties without a label as failures, its balanced accuracy falls from 0.234 to 0.081. This agrees with the finding that verbalised confidence of small models is coarse and poorly informative [94, 93]. Second, the ECE alone would hide all of this: after calibration every system, including the judges, reaches a binary ECE between 0.03 and 0.05, because a calibration map can match the average confidence to the base rate even when the probabilities do not separate the classes. Discrimination must be measured separately, with the AUROC and the risk–coverage curve.

Asking for the probabilities also has a cost. The judge that reads them from logits is as fast as the terse judge, since it generates the same letters, but the judge that writes the JSON object needs 17.0 seconds per answer with fourteen or more decisions on the 3B, as much as the verbose judge, for an accuracy no better than that of the terse one.

7.9 Latency as a function of K

Figure 7.8 shows the central measurement of the thesis: the time needed to verify one answer, on the same 200 answers of T1, as the number of decisions grows. Among the systems that run in process on MLX, GroundOne is the fastest at every size and in every band of K, and its advantage grows with K. For answers with fourteen or more decisions, the 0.5B GroundOne needs 0.41 seconds, against 0.80 for the terse judge and 2.51 for the verbose judge of the same size: it is 1.95 times faster than the first (1.82 to 2.14) and 6.16 times faster than the second (5.72 to 6.67). The 3B GroundOne needs 2.57 seconds, against 4.44 and 15.86, which is 1.73 times (1.65 to 1.82) and 6.18 times (5.88 to 6.65) faster. Reading all verdicts in one pass is also faster than asking one question per sentence in a batch, by a factor of 1.4–1.5 at the highest K. Model size is the other lever: the 0.5B GroundOne verifies an answer about six times faster than the 3B (0.33 against 1.99 seconds at the median).

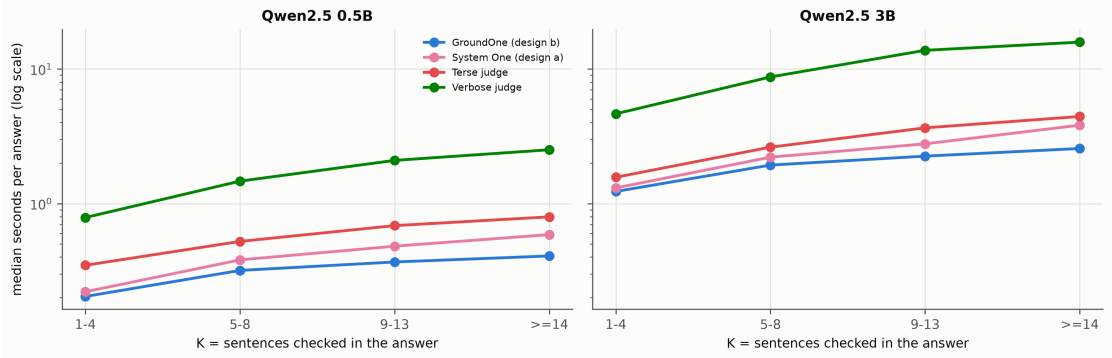


Figure 7.8: Median time per answer as a function of K on the same 200 T1 answers (in-process MLX, one answer at a time, M4 with 16 GB), for GroundOne, the System One with one question per sentence, and the terse and verbose generative judges. Logarithmic scale.

Speed is not the only difference (Figure 7.9). The untrained generative judges are also much less accurate and do not always respect the output format. The 0.5B verbose judge produces a parsable verdict for only 3 % of the sentences, because it mostly copies the answer back, and the 0.5B terse judge fails for 23 %. The 3B judges respect the format far better (0.1 % and 7.6 % failures) but reach a three-way balanced accuracy of only about 0.40, against 0.90 for the 3B GroundOne on the same answers. The verbose 3B judge writes on average 248 tokens per answer, all of which must be generated before the verdict can be used, while GroundOne generates none.

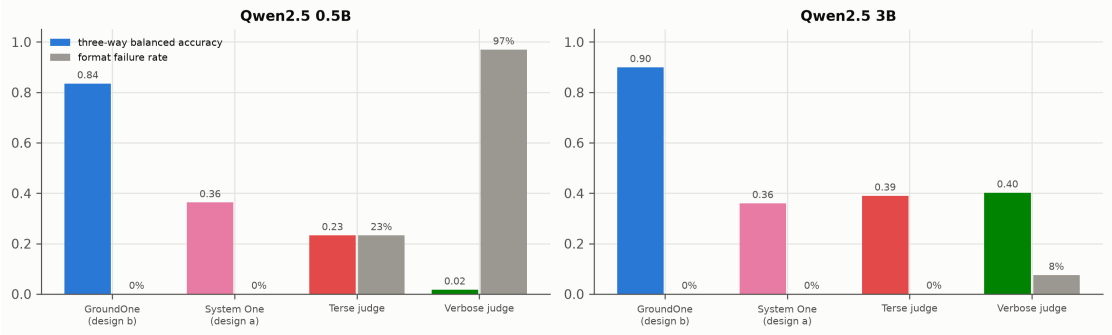


Figure 7.9: On the latency subset: three-way balanced accuracy (format failures counted as errors) and share of sentences whose verdict could not be parsed.

7.10 Energy per verification

The energy measurement adds a qualification that latency alone hides (Figure 7.10). GroundOne uses the least energy at both sizes: 5.5 joules per answer for the 0.5B and 33 for the 3B, against 8.9 and 43 for the terse judge and 22.5 and 112 for the verbose judge. Its advantage, however, is smaller than its advantage in time: 1.6 and 1.3 times less energy than the terse judge, and 4.1 and 3.4 times less than the verbose one. The reason is the power drawn during the computation. GroundOne spends its time in a single long prefill, which keeps the GPU busy and draws about 16 W above idle, while autoregressive decoding is limited by memory bandwidth and draws 11–14 W, but for much longer. This is the behaviour expected from the two phases of transformer inference [40], and it means that a speed-up should never be read as an equal saving of energy [66, 67].

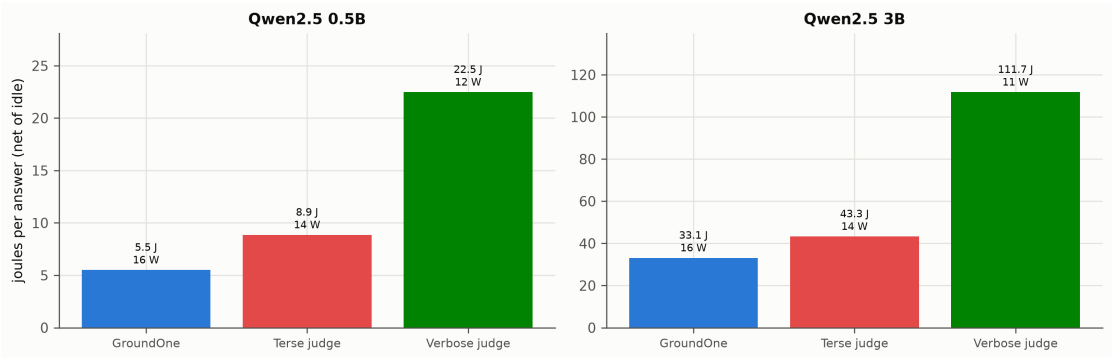


Figure 7.10: Energy per verified answer on 100 latency answers (powermetrics, CPU, GPU and Neural Engine, net of idle power), with the mean net power during the run.

In absolute terms the numbers are small: 5.5 joules is about 0.0015 watt-hours, so the 0.5B GroundOne could verify roughly six hundred answers with the energy of one watt-hour, entirely on the laptop.

7.11 Error analysis

The weak result on VitaminC and the missed contradictions on T1 have a common cause, which word overlap reveals. On VitaminC, when a supported claim shares few words with its evidence (overlap below 0.4) the 0.5B recognises it in only 2 % of cases, against 64 % when the overlap exceeds 0.7, and it never recognises a contradiction in a claim without digits. The models rely partly on lexical overlap: a paraphrased claim looks "not mentioned", and a small change to a copied sentence, often a number, goes unnoticed.

The same weakness exists on T1, with less weight. Where a sentence shares few or some words with the document, the three-way balanced accuracy of the 0.5B is 0.77, but for sentences that copy more than 70 % of their words from the document it falls to 0.54, with a recall of 0.24 for contradicted and 0.43 for not-mentioned sentences. The 3B reduces this blind spot without removing it: on the same sentences its recall is 0.36 and 0.56, and its balanced accuracy 0.63. Hallucinations hidden in sentences that otherwise copy the source remain the main limitation of both models. VitaminC is not harder in absolute terms: it consists mostly of such cases, paraphrased support and minimal edits, which RAGTruth contains in smaller numbers. Appendix A shows the output of every system on the same answer.

Chapter 8

Discussion, Limitations and Threats to Validity

This chapter interprets the results along the five research questions, then states what the experiments cannot show and which choices could have biased them.

8.1 When a specialist System One pays off

Specialisation (RQ1). Training on the decision is what makes GroundOne accurate. The untrained models are near chance, and the same models after training reach 0.769 and 0.832 three-way balanced accuracy on the RAGTruth test set, beating the specialised encoders in AUROC and Rizzo-Flow, a general-purpose System One eight times larger, in three-way accuracy. The advantage is concentrated where the training data are: on LLM-AggreFact the 3B is on par with the best small encoder rather than above it, and below the closed System One Jev and a 27B open model as measured independently; the 0.5B remains below the encoder, and on VitaminC the trained 3B is no better than the untrained one. The comparison with d1-3B, a general-purpose System One of the same size, makes the trade-off explicit. d1-3B, already trained to decide but on a broad mixture of tasks, is far behind GroundOne on RAGTruth, especially on long answers, on par with the 3B on LLM-AggreFact and far ahead on VitaminC. A specialist is therefore a strong choice for the domain it was trained on and a weak general fact checker: specialisation buys accuracy and robustness on long answers in its domain at the cost of transfer, while model size, or a broad training mixture, is what improves transfer outside it.

Readout versus generation (RQ2). The System One mechanism contributes in two separable ways. The first is cost: one pass over the answer replaces the

generation of K verdicts, so the time saved grows with the length of the answer, reaching a factor of about two against a terse judge and six against a verbose one for answers with fourteen or more sentences. The second, which this thesis did not expect, is accuracy. With identical weights and cost, deciding among the three admissible letters is more accurate than taking the token the model would generate, by 0.25 for the 0.5B and 0.07 for the 3B. A fine-tuned System One is better understood as a three-way classifier placed on top of a language model than as a generator that stops after one token, and this distinction matters most for small models, which are the ones a laptop can afford. The comparison with generative judges, finally, also measures training: the judges are untrained, so the large differences in accuracy between them and GroundOne should be read as the combined effect of specialisation and readout.

Calibration (RQ3). A temperature and a bias fitted on development data make the probabilities match the observed frequencies on the test set of the same domain, and the calibrated probabilities make selective prediction effective. Two lessons go beyond GroundOne. First, the expected calibration error alone cannot rank systems: after the same calibration, judges whose probabilities barely separate the classes reach the same ECE as GroundOne, and only the AUROC and the risk–coverage curve reveal the difference. Second, decisions and probabilities must be derived consistently: the most probable calibrated class collapses towards the majority class, while a threshold fitted on development data recovers the balanced decision and keeps the label consistent with the probability shown to the user. Calibration is valid only for the distribution it was fitted on, as the results on VitaminC show.

Cost on the device (RQ4). The verifier fits comfortably in the 16 GB laptop: training peaks at 2.6 and 5.9 GB, and verifying a long answer takes 0.41 seconds with the 0.5B and 2.57 with the 3B, at an energy of 5.5 and 33 joules. The 0.5B meets an interactive budget; the 3B is slower than an interactive target for a typical answer and trades time for accuracy. The energy measurements qualify the speed results: a single long prefill draws more power than token-by-token decoding, so the energy saved is smaller than the time saved, and speed should not be read as an equal saving of battery. The memory needed when the verifier runs next to a generator and a retriever was not measured.

Beyond a binary score (RQ5). The three-way output separates contradicted from not-mentioned sentences well in data-to-text, where RAGTruth concentrates its contradictions, and much less in question answering and summarisation, where contradictions are rare in the training data. The 3B improves this considerably, which suggests that more varied contradictions in training, rather than a different

architecture, are what the small model lacks. Because the strict and lenient policies and the threshold are derived from the same three probabilities, a user can change the notion of error without retraining, which no binary verifier allows.

8.2 Limitations

One platform and one family of models. Every result comes from one laptop, one runtime and one model family. Latency and energy depend on the hardware and on the implementation, and the ratios between methods, not the absolute numbers, are what should transfer. No latency comparison is made with the encoders or the general-purpose System One, because they ran in different runtimes and under conditions that would make the comparison unfair.

Training stability. Two completed runs of the 0.5B agree on all aggregate metrics, but a third run collapsed early and was stopped. The recipe has no gradient clipping and uses Adam without bias correction, both known causes of unstable fine-tuning [98, 97], so either may explain the collapse; neither was tested. The 3B was trained once, on half of the training answers, and its seed variance is unknown.

Training data. RAGTruth is the only training source. Its contradictions are concentrated in data-to-text, and outside that task they are learned only in part. The models also rely partly on lexical overlap: paraphrased support looks unsupported, and a small change in a sentence that otherwise copies the source often goes unnoticed. This blind spot shrinks with model size but remains the main limitation of both models.

Scope of the decision. Verification is sentence by sentence and only against the retrieved document. A sentence that is true in the world but absent from the document counts as not mentioned, which is the right behaviour for groundedness but not for fact checking. Sentences without checkable content are excluded by a rule, and the segmentation itself can split or merge claims differently from a human reader. The verdict on a sentence depends on the document and on the preceding sentences only.

Statistical scope. The bootstrap intervals describe the variability of the test data, not that of training, and about fifty comparisons were tested without correction for multiple testing. Conclusions therefore rest on effects far from zero, and differences on LLM-AggreFact and VitaminC smaller than the spread between seeds are

not claimed. Several baselines were evaluated on subsamples, which widens their intervals.

8.3 Threats to validity

Labels. The distinction between contradicted and not mentioned rests on the RAGTruth annotators, whose agreement on the type of hallucination is not published, and a recent re-annotation of the RAGTruth test split found considerably more hallucinated answers than the original labels [114]. The test set may therefore under-count the minority classes. For this reason the three-way results on RAGTruth are always read together with those on VitaminC, whose labels have measured agreement. LLM-AggreFact unifies the labels of eleven datasets of different quality, and some of its columns are known to be noisy.

In-domain advantage. GroundOne is trained on RAGTruth, so its results on the RAGTruth test set and on the RAGTruth column of LLM-AggreFact favour it with respect to systems trained elsewhere; conversely, VitaminC lies in the training lineage of MiniCheck-RoBERTa. Each comparison is reported with the relevant caveat, and the claims rest on the pattern across the three test sets rather than on a single one.

Baseline configuration. Generative judges are sensitive to their prompts. Their prompts were fixed before looking at test results, and the prompt of the equal-output judge was chosen on development data for format validity only; parsing was lenient in their favour; decoding was not constrained by a grammar, which mostly affects the format failures of the smallest model. A judge fine-tuned on the same data, or a larger judge, was not evaluated, so the comparison measures what a judge of the same size gives without training. Rizzo-Flow was run with 4-bit weights because the 8-bit model did not fit beside the operating system, which may understate its accuracy. The d1-3B was run zero-shot with the questions used for all System One baselines, which may not be the phrasing it works best with, and on a subsample of LLM-AggreFact.

Contamination. The base models were pretrained on large web corpora that may include parts of the public test sets. Because the same base models are used untrained as baselines and reach near-chance accuracy on the three-way task, contamination cannot explain the gains of training, but it cannot be ruled out entirely for the absolute numbers.

One machine. All measurements of time and energy come from one laptop, an Apple M4 with 16 GB, and one runtime, MLX. The cost model of Chapter 2 predicts that the advantage of a System One holds on every class of local hardware, because it rests on the difference between prefill and decoding, which vendors and independent studies observe everywhere; the lower bounds on decoding computed there for other machines follow from published specifications. The actual speed-ups on a discrete graphics card, a DGX Spark or a phone were not measured: their size depends on the balance between computation and bandwidth of each machine, and the direction of the advantage, not its magnitude, is what the cost model supports.

Chapter 9

Conclusion and Future Work

9.1 Summary of findings

This thesis asked whether a compact language model, specialised in a single decision and used as a System One, can verify retrieval-augmented answers sentence by sentence on a laptop, and where any advantage comes from. GroundOne, a fine-tuned Qwen2.5 model that reads the verdicts of a whole answer in one forward pass, gives a positive but qualified answer.

Specialisation works where it was aimed. On the RAGTruth test set GroundOne reaches a three-way balanced accuracy of 0.769 with 0.5B parameters and 0.832 with 3B, far above the same models untrained, above the specialised encoders in ranking supported sentences and well above Rizzo-Flow, a general-purpose System One eight times larger. On LLM-AggreFact the 3B is on par with the best small encoder, and on VitaminC the gain disappears: a specialist is not a general fact checker. d1-3B, a general-purpose System One of the same size trained to decide on a broad mixture of tasks, confirms the trade-off: GroundOne is far ahead of it on RAGTruth, on par on LLM-AggreFact and far behind on VitaminC.

The System One mechanism pays off in two ways. Reading all verdicts in one pass makes verification about two times faster than a terse and six times faster than a verbose generative judge of the same size on long answers, with output that is always valid; and restricting the decision to the admissible answers is, by itself, more accurate than letting the same model generate its verdict. At equal output, generative judges cannot provide probabilities of comparable quality: their probabilities barely separate correct from incorrect decisions, and verbalised ones are almost always all-or-nothing.

The probabilities of GroundOne can be calibrated with a temperature and a bias fitted on development data, and a threshold chosen on the same data turns them into labels that agree with the probabilities and with the evaluated decision. The

expected calibration error alone, however, does not distinguish good probabilities from useless ones, and discrimination must be measured separately.

The whole system runs on a 16 GB laptop. A long answer is verified in 0.41 seconds and 5.5 joules with the 0.5B, and in 2.57 seconds and 33 joules with the 3B. The energy saved with respect to a judge is real but smaller than the time saved, because a single long prefill draws more power than decoding.

The main limitations are the narrow training data, which leave contradictions outside data-to-text and hallucinations hidden in copied sentences only partly learned, and a training recipe that is not fully robust to the random seed.

9.2 Future work

Several directions follow directly from these limits. The training recipe should be made robust, by enabling bias correction and gradient clipping, logging gradient norms and repeating the runs over several seeds, including for the 3B. Training data with more varied contradictions, and with paraphrased support and minimal edits of the kind VitaminC contains, should address both the weak recall outside data-to-text and the blind spot on copied sentences. Training the readout on the letter that the model would naturally write after the marker could align readout and generation, with effects on accuracy that remain to be measured.

On the systems side, the cost model of Chapter 2 should be checked on other hardware: a port of the readout to llama.cpp would allow measuring GroundOne on a discrete graphics card, on a DGX Spark or an AMD Strix Halo machine, and on a phone, whose neural processing units accelerate exactly the prefill on which a System One relies. The remaining measurements of the locality pillar are the memory used when the verifier runs next to a generator and a retriever, and a complete evaluation with the network disabled. A fair latency comparison with the encoders requires running them in batches, and one with the general-purpose System One models requires comparing both systems through the same HTTP interface. Verification could also run while the answer is being generated, flagging each sentence as soon as it is complete, which the marker design makes natural. Finally, a local assistant built on the served interface would let users choose the policy and the threshold that suit their documents, which is where calibrated probabilities show their value.

Appendix A

Output of every system on one answer

This appendix shows what each system returns for the same answer of T1, so that the numbers of Chapter 7 can be related to concrete outputs. The answer (identifier `rt-7784`) is a data-to-text answer written by a language model from a structured description of a restaurant: a JSON record with its name, address, categories, opening hours (Tuesday to Sunday, 11:00–21:00), attributes, an average rating of 4.5 stars and two customer reviews. It was chosen because every system scored it and because it contains all three classes; it was not selected for the performance of any system. Its six sentences and their reference labels are:

1. *Mesa Verde is a popular business located in Santa Barbara, CA.* — supported.

2. *It offers a variety of services including Garage Door Services, Restaurants, Live/Raw Food, . . . , Vegan, and Salad.* — supported.

3. *The business is open seven days a week from 11 am to 9 pm.* — contradicted (it is closed on Mondays).

4. *Mesa Verde has a rating of 4.5 stars on Google and has received multiple positive reviews.* — not mentioned (the record gives no source for the rating and contains two reviews, one of them negative).

5. *One customer praised the restaurant's fresh, delicious, and inventive cuisine, while another was disappointed by the taste and price of the food.* — supported.

6. *Despite this, many customers have had positive experiences at Mesa Verde and recommend it for its fusion and vegan food options.* — supported.

A.1 Verdicts sentence by sentence

Table A.1 lists the verdict of every system. For GroundOne the label is the served one, derived from the calibrated probabilities with the threshold fitted on development data ($\tau = 0.935$ for the 0.5B and 0.925 for the 3B), and the number is the calibrated P(supported). For the encoders the number is their probability of support and the decision is taken at 0.5. S, C and N stand for supported, contradicted and not mentioned; a dash marks a format failure.

Table A.1: Verdicts on answer `rt-7784`. Errors with respect to the reference are in bold.

System	1	2	3	4	5	6
Reference	S	S	C	N	S	S
GroundOne 0.5B (served, P_{sup})	S 0.96	S 0.98	C 0.50	N 0.05	S 0.98	S 0.97
GroundOne 3B (served, P_{sup})	S 0.96	S 0.99	C 0.58	N 0.21	S 0.99	**N 0.85**
B2 0.5B (generated token)	S	S	C	N	S	–
B2 3B (generated token)	S	S	C	–	S	S
Untrained 0.5B (readout)	**C**	**C**	C	**C**	**C**	**C**
Terse judge 0.5B	S	S	**S**	**S**	S	S
Terse judge 3B	**C**	S	**S**	**S**	**N**	S
Verbose judge 0.5B	–	–	–	–	–	–
Verbose judge 3B	–	–	–	–	–	–
Equal-output judge 3B, logits	**C**	S	**S**	**S**	**N**	S
Equal-output judge 3B, verbalised	S	**C**	**N**	**S**	**C**	**N**
MiniCheck-RoBERTa (P_{sup})	0.78	0.90	0.09	0.03	**0.43**	**0.03**
HHEM-2.1-open (P_{sup})	0.75	0.95	0.01	0.09	0.73	**0.24**
Rizzo-Flow 4B	S	S	**S**	**S**	S	S

The 0.5B GroundOne gets all six sentences right. The contradiction in sentence 3 is caught with a calibrated P(supported) of 0.50: well below the threshold, but far from certain, which is what a calibrated probability should express for a sentence that is correct except for one day of the week. The 3B GroundOne also catches both errors but serves sentence 6, a generalisation over the two reviews, as not mentioned: its P(supported) of 0.85 is below the high threshold that the base rate of RAGTruth imposes, a case where a user who prefers fewer flags would lower the threshold. With the same weights, the generated token (B2) gives the right letter on five of the six sentences, but on the remaining one it produces a space (0.5B) or the word “We” (3B) instead of a verdict. The binary encoders separate the two errors well but also reject sentences 5 or 6, which summarise the reviews in different words; Rizzo-Flow accepts every sentence. The generative judges fail in the ways that Chapter 7 quantifies, as the raw outputs below show.

A.2 Raw outputs

GroundOne 0.5B (served JSON, abridged). One forward pass over 1,193 input tokens, no generated token, 0.47 seconds (2.6 seconds for the 3B):


```
{"design": "b", "threshold": 0.935, "n_decisions": 6,
 "usage": {"input_tokens": 1193, "output_tokens": 0},
 "sentences": [
  {"index": 2, "text": "The business is open seven days a week ...",
   "scored": true, "labels": {"three": "contradicted"},
   "answers": {"three": {"supported": 0.502, "contradicted": 0.462,
                         "not_mentioned": 0.036}},
   "flag_strict": true, "flag_lenient": true},
  {"index": 3, "text": "Mesa Verde has a rating of 4.5 stars ...",
   "scored": true, "labels": {"three": "not_mentioned"},
   "answers": {"three": {"supported": 0.051, "contradicted": 0.003,
                         "not_mentioned": 0.946}},
   "flag_strict": true, "flag_lenient": false}, ...]}
```


Terse judge 3B. One line per sentence, as requested, but with the wrong letters for four sentences:

```
1: B
2: A
3: A
4: A
5: C
6: A
```

Verbose judge 3B (first three verdicts). The reasoning restates the sentence and cites the document as support even for the contradicted and the unsupported sentence, and every verdict is "A". The verdicts are written on a separate line instead of on the same line, as the instruction asked, so the protocol counts them as format failures:

```
1: Mesa Verde is located in Santa Barbara, CA, as stated in the document.
Verdict: A

2: Mesa Verde offers a variety of services including Garage Door Services,
   [...], as mentioned in the categories provided.
Verdict: A

3: The business is open seven days a week from 11 am to 9 pm, as indicated
   by the hours provided.
Verdict: A
```

Verbose judge 0.5B. It copies the six sentences back, numbered, without any verdict.

Equal-output judge 3B, verbalised. The probabilities are all one-hot, and for sentences 3 and 6 the stated label (C) contradicts the stated probabilities, which put all the mass on A:


```
{"1": {"label": "A", "p": [1.0, 0.0, 0.0]},
 "2": {"label": "B", "p": [0.0, 1.0, 0.0]},
 "3": {"label": "C", "p": [1.0, 0.0, 0.0]},
 "4": {"label": "A", "p": [1.0, 0.0, 0.0]},
 "5": {"label": "B", "p": [0.0, 1.0, 0.0]},
 "6": {"label": "C", "p": [1.0, 0.0, 0.0]}}
```


Equal-output judge 0.5B, verbalised. The same vector for every sentence:


```
{"1": {"label": "A", "p": [0.3, 0.3, 0.3]},
 "2": {"label": "A", "p": [0.3, 0.3, 0.3]},
 ...
 "6": {"label": "A", "p": [0.3, 0.3, 0.3]}}
```

Bibliography

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2020 (cit. on pp. 1, 31).

[2] Georgi Gerganov and ggml-org contributors. *llama.cpp: LLM inference in C/C++*. 2023. URL: `https://github.com/ggml-org/llama.cpp` (cit. on pp. 1, 24, 26, 31, 50).

[3] Apple. *MLX: An array framework for Apple silicon*. 2023. URL: `https://github.com/ml-explore/mlx` (cit. on pp. 1, 24, 38).

[4] Tom Gunter, Zirui Wang, Chong Wang, Ruoming Pang, et al. *Apple Intelligence Foundation Language Models*. 2024. arXiv: `2407.21075` (cit. on pp. 1, 11, 14).

[5] Venkat Somala and Luke Emberson. *Frontier AI performance becomes accessible on consumer hardware within a year*. Epoch AI. Aug. 15, 2025. URL: `https://epoch.ai/data-insights/consumer-gpu-model-gap` (visited on 2026-10-11) (cit. on pp. 1, 13).

[6] Stanford Institute for Human-Centered Artificial Intelligence. *The 2025 AI Index Report: Technical Performance*. 2025. URL: `https://hai.stanford.edu/ai-index/2025-ai-index-report/technical-performance` (visited on 2026-10-11) (cit. on pp. 1, 13).

[7] NYU Shanghai RITS. *OpenAI claims a Navier–Stokes proof, amid a dispute over credit*. Sept. 9, 2026. URL: `https://rits.shanghai.nyu.edu/ai/openai-navier-stokes-proof-credit-dispute/` (visited on 2026-10-11) (cit. on pp. 1, 10).

[8] Cheng Niu, Yuanhao Wu, Juno Zhu, Siliang Xu, et al. "RAGTruth: A Hallucination Corpus for Developing Trustworthy Retrieval-Augmented Language Models". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. on pp. 2, 5, 31, 40, 41).

[9] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, et al. "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions". In: *ACM Transactions on Information Systems* (2025). DOI: 10.1145/3703155 (cit. on pp. 2, 31).

[10] Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, et al. "Efficiently Scaling Transformer Inference". In: *Proceedings of Machine Learning and Systems (MLSys)*. 2023 (cit. on pp. 3, 15, 16, 35).

[11] Yuheng Zha, Yichi Yang, Ruichen Li, and Zhiting Hu. "AlignScore: Evaluating Factual Consistency with a Unified Alignment Function". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023 (cit. on pp. 3, 32, 60).

[12] Liyan Tang, Philippe Laban, and Greg Durrett. "MiniCheck: Efficient Fact-Checking of LLMs on Grounding Documents". In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2024 (cit. on pp. 3, 32, 40, 41, 51).

[13] Vectara. *HHEM-2.1-Open: Hughes Hallucination Evaluation Model*. 2024. URL: https://huggingface.co/vectara/hallucination_evaluation_model (cit. on pp. 3, 32, 51).

[14] TypeSafe. *Introducing System One Models and Jev*. Blog post. 2026. URL: https://typesafe.ai/blog/introducing-system-one-models-and-jev (cit. on pp. 3, 29).

[15] TheoLeeCJ. *SemIf: open option-logit System One baseline*. 2026. URL: https://github.com/TheoLeeCJ/SemIf (cit. on pp. 3, 29).

[16] Simone Rizzo. *Rizzo Flow: typed decisions from an LLM, without generating a single token*. 2026. URL: https://github.com/Rizzo-AI-Academy/rizzo-flow (cit. on pp. 3, 30, 50).

[17] Hugging Face. *Models tagged decision-model*. URL: https://huggingface.co/models?other=decision-model (visited on 2026-10-10) (cit. on pp. 4, 30).

[18] Liquid AI. *d1-3B: a decision model built on LFM2.5-VL-3B (model card, LiquidAI/d1-3B, revision 051bcc4)*. URL: https://huggingface.co/LiquidAI/d1-3B (visited on 2026-10-10) (cit. on pp. 4, 30, 50).

[19] Cloudflare. *Clef: open decision models (model card, Cloudflare/clef)*. URL: https://huggingface.co/Cloudflare/clef (visited on 2026-10-10) (cit. on pp. 4, 30).

[20] Tobias Deußer, Lorenz Sparrenberg, and Rafet Sifa. *Evaluating and Benchmarking the System One Model Jev*. 2026. arXiv: 2609.37647 (cit. on pp. 5, 30, 59).

[21] Gowthamkumar Nandakishore. *Laya as a Typed Probabilistic Assessor: An Independent Reproduction and a Preregistered Study of Calibration and Selective Escalation*. 2026. arXiv: 2609.33843 (cit. on pp. 5, 30).

[22] multimodalart. *Jev Decision Index: a leaderboard of System One decision models (Hugging Face Space, v0.2.1)*. URL: https://huggingface.co/spaces/multimodalart/jev-decision-index (visited on 2026-10-10) (cit. on pp. 5, 30, 57).

[23] Qwen Team. *Qwen2.5 Technical Report*. 2024. arXiv: 2412.15115 (cit. on pp. 5, 13, 33, 36, 48).

[24] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. "LoRA: Low-Rank Adaptation of Large Language Models". In: *International Conference on Learning Representations (ICLR)*. 2022 (cit. on pp. 5, 33, 36).

[25] Guanqiao Qu, Qiyuan Chen, Wei Wei, Zheng Lin, Xianhao Chen, and Kaibin Huang. "Mobile Edge Intelligence for Large Language Models: A Contemporary Survey". In: *IEEE Communications Surveys & Tutorials* 27.6 (2025), pp. 3820–3860. arXiv: 2407.18921 (cit. on pp. 10, 11).

[26] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)*. Official Journal of the European Union, L 119, 4 May 2016, pp. 1–88. 2016. URL: https://eur-lex.europa.eu/eli/reg/2016/679/oj (cit. on p. 10).

[27] Jeremy Kahn. *OpenAI says it cracked a Navier–Stokes math grand challenge; a mathematician raises accusations (Eye on AI)*. Fortune. Sept. 8, 2026. URL: https://fortune.com/2026/09/08/openai-says-it-cracked-navier-stokes-math-grand-challenge-buckmaster-accusation-cheating-intimidation-tao-lament/ (visited on 2026-10-11) (cit. on p. 10).

[28] Larissa G. Capella. *'Why would you ruin your career?': OpenAI claims to have cracked one of math's greatest unsolved problems, but mathematicians allege it played dirty*. Live Science. Sept. 9, 2026. URL: https://livescience.com/physics-mathematics/mathematics/why-would-you-ruin-your-career-openai-claims-to-have-cracked-one-of-maths-greatest-unsolved-problems-but-mathematicians-allege-it-played-dirty (visited on 2026-10-11) (cit. on p. 10).

[29] Apple. *Private Cloud Compute: A new frontier for AI privacy in the cloud*. June 10, 2024. URL: https://security.apple.com/blog/private-cloud-compute/ (visited on 2026-10-11) (cit. on pp. 10, 11).

[30] Senyao Li, Haozhao Wang, Wenchao Xu, Rui Zhang, Song Guo, Jingling Yuan, Xian Zhong, Tianwei Zhang, and Ruixuan Li. *Collaborative Inference and Learning between Edge SLMs and Cloud LLMs: A Survey of Algorithms, Execution, and Open Challenges*. 2025. arXiv: 2507.16731 (cit. on pp. 11, 21).

[31] Open Source Initiative. *The Open Source AI Definition, version 1.0*. Oct. 28, 2024. URL: https://opensource.org/ai/open-source-ai-definition (visited on 2026-10-11) (cit. on p. 12).

[32] Artificial Analysis. *Introducing the Artificial Analysis Openness Index*. Dec. 1, 2025. URL: https://artificialanalysis.ai/articles/announcing-artificial-analysis-openness-index (visited on 2026-10-11) (cit. on p. 13).

[33] Rishi Bommasani, Alice Hau, Kevin Klyman, and Percy Liang. *Foundation models under the EU AI Act*. Stanford Center for Research on Foundation Models. Aug. 1, 2024. URL: https://crfm.stanford.edu/2024/08/01/eu-ai-act.html (visited on 2026-10-11) (cit. on p. 13).

[34] Epoch AI. *Open-weight models lag the closed frontier by about four months on the Epoch Capabilities Index*. May 29, 2026. URL: https://epoch.ai/data-insights/open-closed-eci-gap (visited on 2026-10-11) (cit. on pp. 13, 14).

[35] Venkat Somala and Luke Emberson. *What's the best model you can run on a single consumer GPU?* Epoch AI. Sept. 9, 2025. URL: https://epochai.substack.com/p/whats-the-best-model-you-can-run (visited on 2026-10-11) (cit. on p. 13).

[36] Zhenyan Lu, Xiang Li, Dongqi Cai, Rongjie Yi, Fangming Liu, Xiwen Zhang, Nicholas D. Lane, and Mengwei Xu. *Small Language Models: Survey, Measurements, and Insights*. 2024. arXiv: 2409.15790 (cit. on pp. 13, 14).

[37] Zechun Liu et al. "MobileLLM: Optimizing Sub-billion Parameter Language Models for On-Device Use Cases". In: *Proceedings of the 41st International Conference on Machine Learning (ICML)*. Vol. 235. PMLR. 2024 (cit. on pp. 14, 27).

[38] Google. *Introducing Gemma 3n: the developer guide*. May 20, 2025. URL: https://developers.googleblog.com/en/introducing-gemma-3n/ (visited on 2026-10-11) (cit. on p. 14).

[39] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. 2023 (cit. on p. 15).

[40] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, et al. "Splitwise: Efficient Generative LLM Inference Using Phase Splitting". In: *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*. 2024 (cit. on pp. 15, 20, 27, 35, 65).

[41] Samuel Williams, Andrew Waterman, and David Patterson. "Roofline: An Insightful Visual Performance Model for Multicore Architectures". In: *Communications of the ACM* 52.4 (2009), pp. 65–76. DOI: 10.1145/1498765.1498785 (cit. on pp. 16, 17).

[42] Zhihang Yuan et al. *LLM Inference Unveiled: Survey and Roofline Model Insights*. 2024. arXiv: 2402.16363 (cit. on p. 16).

[43] Apple. *MacBook Pro (14-inch, M4, 2024) – Technical Specifications*. URL: https://support.apple.com/en-us/121552 (visited on 2026-10-10) (cit. on pp. 16, 21, 48).

[44] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, Karen Khatamifard, et al. "LLM in a flash: Efficient Large Language Model Inference with Limited Memory". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. on pp. 17, 25).

[45] Apple. *MacBook Pro – Technical Specifications (M5, M5 Pro and M5 Max)*. URL: https://www.apple.com/macbook-pro/specs/ (visited on 2026-10-11) (cit. on p. 21).

[46] Apple. *Mac mini – Technical Specifications*. URL: https://www.apple.com/mac-mini/specs/ (visited on 2026-10-11) (cit. on pp. 19, 21).

[47] Apple. *Mac Studio (2025) – Technical Specifications*. URL: https://support.apple.com/en-us/122211 (visited on 2026-10-11) (cit. on pp. 19, 21).

[48] NVIDIA. *DGX Spark User Guide: Hardware Overview*. Sept. 10, 2026. URL: https://docs.nvidia.com/dgx/dgx-spark/hardware.html (visited on 2026-10-11) (cit. on pp. 20, 21).

[49] Mark Hachman. *Framework Desktop review*. PCWorld. Aug. 7, 2025. URL: https://www.pcworld.com/article/2866400/framework-desktop-review.html (visited on 2026-10-11) (cit. on pp. 19, 21).

[50] NVIDIA. *Compare GeForce Graphics Cards: specifications*. URL: https://www.nvidia.com/en-us/geforce/graphics-cards/compare/ (visited on 2026-10-11) (cit. on pp. 20, 21).

[51] EXO Labs. *NVIDIA DGX Spark and Mac Studio: disaggregated prefill and decode*. 2025. URL: https://blog.exolabs.net/nvidia-dgx-spark/ (visited on 2026-10-11) (cit. on p. 20).

[52] Zhenliang Xue, Yixin Song, Zeyu Mi, Xinrui Zheng, Yubin Xia, and Haibo Chen. *PowerInfer-2: Fast Large Language Model Inference on a Smartphone*. 2024. arXiv: 2406.06282 (cit. on p. 21).

[53] Daliang Xu, Hao Zhang, Liming Yang, Ruiqi Liu, Gang Huang, Mengwei Xu, and Xuanzhe Liu. "Fast On-device LLM Inference with NPUs". In: *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '25)*. ACM, 2025. DOI: 10.1145/3669940.3707239 (cit. on pp. 21, 22, 27).

[54] Apple Machine Learning Research. *Exploring LLMs with MLX and the Neural Accelerators in the M5 GPU*. Nov. 19, 2025. URL: https://machinelearning.apple.com/research/exploring-llms-mlx-m5 (visited on 2026-10-11) (cit. on p. 22).

[55] Subhransu Das, Jiaming Cheng, Arnav Kumar, Sadia Afrose, Mingzhe Han, Michael Silagy, Shreya Palande, Brijesh Soni, and Rajiv Ramnath. *Large Models for Small Devices: Edge AI Deployment*. 2026. arXiv: 2608.15693 (cit. on p. 22).

[56] Apple. *mlx-lm: LLMs with MLX*. 2024. URL: https://github.com/ml-explore/mlx-lm (cit. on pp. 24, 26, 38).

[57] Varun Rajesh, Om Jodhpurkar, Pooja Anbuselvan, Mantinder Singh, Ashok Jallepali, Shantanu Godbole, Pradeep Kumar Sharma, and Hritvik Shrivastava. *Production-Grade Local LLM Inference on Apple Silicon: A Comparative Study of MLX, MLC-LLM, Ollama, llama.cpp, and PyTorch MPS*. 2025. arXiv: 2511.05502 (cit. on p. 24).

[58] Stefanos Laskaridis, Kleomenis Katevas, Lorenzo Minto, and Hamed Haddadi. "MELTing Point: Mobile Evaluation of Language Transformers". In: *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom)*. 2024. arXiv: 2403.12844 (cit. on pp. 24, 26, 27).

[59] Salvatore Sanfilippo. *DwarfStar (ds4): DeepSeek 4 Flash and PRO local inference engine for Metal, CUDA and ROCm*. URL: https://github.com/antirez/ds4 (visited on 2026-10-13) (cit. on p. 24).

[60] JustVugg. *colibri: run frontier MoE models on hardware you already own, experts streamed from disk*. URL: https://github.com/JustVugg/colibri (visited on 2026-10-13) (cit. on pp. 24, 25).

[61] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 35. 2022 (cit. on p. 26).

[62] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers". In: *International Conference on Learning Representations (ICLR)*. 2023 (cit. on p. 26).

[63] Ji Lin et al. "AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration". In: *Proceedings of Machine Learning and Systems (MLSys)*. 2024 (cit. on p. 26).

[64] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. "QLoRA: Efficient Finetuning of Quantized LLMs". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2023 (cit. on pp. 26, 33, 36).

[65] Yaniv Leviathan, Matan Kalman, and Yossi Matias. "Fast Inference from Transformers via Speculative Decoding". In: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. Vol. 202. PMLR. 2023 (cit. on p. 27).

[66] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, et al. "From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference". In: *IEEE High Performance Extreme Computing Conference (HPEC)*. 2023 (cit. on pp. 27, 65).

[67] Alexandra Sasha Luccioni, Yacine Jernite, and Emma Strubell. "Power Hungry Processing: Watts Driving the Cost of AI Deployment?" In: *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency (FAccT)*. 2024 (cit. on pp. 27, 65).

[68] Jacky Kwok, Hangoo Kang, Tarun Suresh, Jon Saad-Falcon, Marco Pavone, Christopher Ré, and Azalia Mirhoseini. *Contrastive Language Models: A System One Model for Fast and Generalizable Decision-Making*. Blog post, not peer-reviewed. 2026. URL: `https://contrastive-lm.notion.site` (cit. on p. 30).

[69] Convai Innovations. *Laya: a System One decision model (model card, convaiinnovations/laya)*. URL: `https://huggingface.co/convaiinnovations/laya` (visited on 2026-10-10) (cit. on p. 30).

[70] Seyedarmin Azizi, Erfan Baghaei Potraghloo, and Massoud Pedram. *Labels Override Definitions in Jev-Style Typed Decision Models*. 2026. arXiv: `2610.02586` (cit. on p. 30).

[71] Cho Yin Yong. *DecideBench (dataset card, choyiny/decidebench)*. URL: `https://huggingface.co/datasets/choyiny/decidebench` (visited on 2026-10-10) (cit. on p. 30).

[72] Perplexity. *pplx-decider-v1.1-27b (model card)*. URL: https://huggingface.co/perplexity-ai/pplx-decider-v1.1-27b (visited on 2026-10-10) (cit. on p. 30).

[73] Ruoqi Guo, Yi Liu, Gelei Deng, Yuekang Li, et al. *Just Ask Jev: Reinforcement Learning for Calibrated Decisions as a Zero-Shot Detector of AI Alignment Failures*. 2026. arXiv: 2609.29429 (cit. on p. 30).

[74] Yubo Li, Yidi Miao, Ramayya Krishnan, and Rema Padman. *JEV-as-a-Judge: Accept When Confident, Escalate When Unsure*. 2026. arXiv: 2609.26550 (cit. on p. 30).

[75] Xinpeng Wang, Bolei Ma, Chengzhi Hu, Leon Weber-Genzel, et al. ""My Answer is C": First-Token Probabilities Do Not Match Text Answers in Instruction-Tuned Language Models". In: *Findings of the Association for Computational Linguistics: ACL 2024*. 2024. arXiv: 2402.14499 (cit. on pp. 31, 36).

[76] Chujie Zheng, Hao Zhou, Fandong Meng, Jie Zhou, and Minlie Huang. "Large Language Models Are Not Robust Multiple Choice Selectors". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. on pp. 31, 36).

[77] Adina Williams, Nikita Nangia, and Samuel R. Bowman. "A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference". In: *Proceedings of NAACL-HLT 2018*. 2018 (cit. on p. 31).

[78] Yixin Nie, Adina Williams, Emily Dinan, Mohit Bansal, Jason Weston, and Douwe Kiela. "Adversarial NLI: A New Benchmark for Natural Language Understanding". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020 (cit. on pp. 31, 41).

[79] Tal Schuster, Adam Fisch, and Regina Barzilay. "Get Your Vitamin C! Robust Fact Verification with Contrastive Evidence". In: *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2021, pp. 624–643. DOI: 10.18653/v1/2021.naacl-main.52. URL: https://aclanthology.org/2021.naacl-main.52 (cit. on pp. 31, 40, 41).

[80] Philippe Laban, Tobias Schnabel, Paul N. Bennett, and Marti A. Hearst. "SummaC: Re-Visiting NLI-based Models for Inconsistency Detection in Summarization". In: *Transactions of the Association for Computational Linguistics* 10 (2022), pp. 163–177 (cit. on p. 32).

[81] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, et al. "FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation". In: *Proceedings of EMNLP 2023*. 2023 (cit. on p. 32).

[82] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, et al. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena". In: *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*. 2023 (cit. on p. 32).

[83] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. "RAGAs: Automated Evaluation of Retrieval Augmented Generation". In: *Proceedings of EACL 2024: System Demonstrations*. 2024 (cit. on p. 32).

[84] Jon Saad-Falcon, Omar Khattab, Christopher Potts, and Matei Zaharia. "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems". In: *Proceedings of NAACL 2024*. 2024 (cit. on p. 32).

[85] Selvan Sunitha Ravi, Bartosz Mielczarek, Anand Kannappan, Douwe Kiela, and Rebecca Qian. *Lynx: An Open Source Hallucination Evaluation Model*. 2024. arXiv: 2407.08488 (cit. on p. 32).

[86] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. on p. 32).

[87] Brandon T. Willard and Rémi Louf. *Efficient Guided Generation for Large Language Models*. 2023. arXiv: 2307.09702 (cit. on pp. 32, 50).

[88] Liyan Tang, Philippe Laban, and Greg Durrett. *LLM-AggreFact Leaderboard*. 2024. URL: https://llm-aggrefact.github.io/ (visited on 2026-10-05) (cit. on pp. 32, 40, 41).

[89] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. "On Calibration of Modern Neural Networks". In: *International Conference on Machine Learning (ICML)*. 2017 (cit. on pp. 32, 37).

[90] Mahdi Pakdaman Naeini, Gregory F. Cooper, and Milos Hauskrecht. "Obtaining Well Calibrated Probabilities Using Bayesian Binning". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2015 (cit. on pp. 32, 37).

[91] Tony Z. Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. "Calibrate Before Use: Improving Few-Shot Performance of Language Models". In: *International Conference on Machine Learning (ICML)*. 2021 (cit. on pp. 32, 37).

[92] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, et al. *Language Models (Mostly) Know What They Know*. 2022. arXiv: 2207.05221 (cit. on p. 32).

[93] Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, et al. "Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback". In: *Proceedings of EMNLP 2023*. 2023 (cit. on pp. 32, 63).

[94] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, et al. "Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs". In: *International Conference on Learning Representations (ICLR)*. 2024 (cit. on pp. 32, 63).

[95] Yonatan Geifman and Ran El-Yaniv. "Selective Classification for Deep Neural Networks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2017 (cit. on pp. 32, 51).

[96] Jesse Dodge, Gabriel Ilharco, Roy Schwartz, Ali Farhadi, Hannaneh Hajishirzi, and Noah Smith. *Fine-Tuning Pretrained Language Models: Weight Initializations, Data Orders, and Early Stopping*. 2020. arXiv: 2002.06305 (cit. on pp. 33, 48).

[97] Marius Mosbach, Maksym Andriushchenko, and Dietrich Klakow. "On the Stability of Fine-tuning BERT: Misconceptions, Explanations, and Strong Baselines". In: *International Conference on Learning Representations (ICLR)*. 2021 (cit. on pp. 33, 56, 69).

[98] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. "On the Difficulty of Training Recurrent Neural Networks". In: *International Conference on Machine Learning (ICML)*. 2013 (cit. on pp. 33, 56, 69).

[99] Tri Nguyen, Mir Rosenberg, Xia Song, Jianfeng Gao, Saurabh Tiwary, Rangan Majumder, and Li Deng. *MS MARCO: A Human Generated MAchine Reading COmprehension Dataset*. CoCo Workshop at NIPS 2016. 2016. arXiv: 1611.09268 (cit. on p. 40).

[100] Karl Moritz Hermann, Tomáš Kočiský, Edward Grefenstette, Lasse Espeholt, Will Kay, Mustafa Suleyman, and Phil Blunsom. "Teaching Machines to Read and Comprehend". In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2015 (cit. on p. 40).

[101] Abigail See, Peter J. Liu, and Christopher D. Manning. "Get To The Point: Summarization with Pointer-Generator Networks". In: *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2017 (cit. on p. 40).

[102] Yelp. *Yelp Open Dataset*. URL: https://business.yelp.com/data/resources/open-dataset/ (visited on 2026-10-05) (cit. on p. 40).

[103] Liyan Tang, Tanya Goyal, Alex Fabbri, Philippe Laban, Jiacheng Xu, Semih Yavuz, Wojciech Kryściński, Justin Rousseau, and Greg Durrett. "Understanding Factual Errors in Summarization: Errors, Summarizers, Datasets, Error Detectors". In: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*. 2023 (cit. on p. 41).

[104] Liyan Tang, Igor Shalyminov, Amy Wing-mei Wong, et al. "TofuEval: Evaluating Hallucinations of LLMs on Topic-Focused Dialogue Summarization". In: *Proceedings of NAACL 2024*. 2024 (cit. on p. 41).

[105] Nelson F. Liu, Tianyi Zhang, and Percy Liang. "Evaluating Verifiability in Generative Search Engines". In: *Findings of EMNLP 2023*. 2023 (cit. on p. 41).

[106] Chaitanya Malaviya, Subin Lee, Sihao Chen, Elizabeth Sieber, Mark Yatskar, and Dan Roth. "ExpertQA: Expert-Curated Questions and Attributed Answers". In: *Proceedings of NAACL 2024*. 2024 (cit. on p. 41).

[107] Hung-Ting Chen, Fangyuan Xu, Shane Arora, and Eunsol Choi. "Understanding Retrieval Augmentation for Long-Form Question Answering". In: *Conference on Language Modeling (COLM)*. 2024 (cit. on p. 41).

[108] Ryo Kamoi, Tanya Goyal, Juan Diego Rodriguez, and Greg Durrett. "WiCE: Real-World Entailment for Claims in Wikipedia". In: *Proceedings of EMNLP 2023*. 2023 (cit. on p. 41).

[109] Alon Jacovi, Yonatan Bitton, Bernd Bohnet, Jonathan Herzig, Or Honovich, Michael Tseng, Michael Collins, Roee Aharoni, and Mor Geva. "A Chain-of-Thought Is as Strong as Its Weakest Link: A Benchmark for Verifiers of Reasoning Chains". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. 2024 (cit. on p. 41).

[110] Yuxia Wang, Revanth Gangi Reddy, Zain Muhammad Mujahid, et al. "Factcheck-Bench: Fine-Grained Evaluation Benchmark for Automatic Fact-checkers". In: *Findings of EMNLP 2024*. 2024 (cit. on p. 41).

[111] Liyan Tang. *LLM-AggreFact dataset card (lytang/LLM-AggreFact)*. URL: `https://huggingface.co/datasets/lytang/LLM-AggreFact` (visited on 2026-10-05) (cit. on p. 41).

[112] Tal Schuster. *VitaminC dataset card (tals/vitaminc)*. URL: `https://huggingface.co/datasets/tals/vitaminc` (visited on 2026-10-06) (cit. on p. 41).

[113] Nipun Sadvilkar and Mark Neumann. "PySBD: Pragmatic Sentence Boundary Disambiguation". In: *Proceedings of the Second Workshop for NLP Open Source Software (NLP-OSS)*. 2020, pp. 110–114. DOI: 10.18653/v1/2020.nlposs-1.15. URL: https://aclanthology.org/2020.nlposs-1.15 (cit. on p. 41).

[114] Boxi Yu, Yuzhong Zhang, Liting Lin, Lionel Briand, and Emir Muñoz. *Retromorphic Testing with Hierarchical Verification for Hallucination Detection in RAG*. 2026. arXiv: 2603.27752. URL: https://arxiv.org/abs/2603.27752 (cit. on pp. 45, 70).

[115] Philipp Koehn. "Statistical Significance Tests for Machine Translation Evaluation". In: *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2004, pp. 388–395. URL: https://aclanthology.org/W04-3250 (cit. on p. 51).

[116] Taylor Berg-Kirkpatrick, David Burkett, and Dan Klein. "An Empirical Investigation of Statistical Significance in NLP". In: *Proceedings of EMNLP-CoNLL 2012*. 2012, pp. 995–1005. URL: https://aclanthology.org/D12-1091 (cit. on p. 51).

[117] Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. "The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing". In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2018, pp. 1383–1392. URL: https://aclanthology.org/P18-1128 (cit. on p. 51).